WealthWise

An Intelligent Portfolio Analytics and Goal-Planning Platform
for Indian Mutual Fund Investors

Comprehensive Project Documentation

Project Synopsis • SRS • System Design • API Docs • Deployment Guide • User Manual • Testing • Azure Deployment

Spring Boot 3.2 • React 19 • PostgreSQL 15 • Docker • Microsoft Azure

April 2026

Table of Contents

1. Project Synopsis
   Abstract, Problem Statement, Objectives, Scope, Technology Stack
2. Software Requirements Specification (SRS)
   30 Functional Requirements, 30 Non-Functional Requirements, Feature Matrix, 4 Use Cases
3. System Design — High-Level Design (HLD)
   Architecture Overview, Technology Justification, Caching, Security Architecture
4. System Design — Low-Level Design (LLD)
   Class Diagram, 3 Sequence Diagrams, DFD Level 0/1, ER Diagram, Algorithm Pseudocode
5. API Documentation
   37 Endpoints across 9 API groups, JWT authentication, Request/Response samples
6. Deployment Guide
   Prerequisites, Environment Variables, Local Setup, Docker, Render Deployment, Troubleshooting
7. User Manual
   Feature Guide, Navigation, Screen Descriptions, 15 FAQs
8. Testing Documentation
   280 Automated Tests, 15 Test Suites, Testcontainers, MSW, Playwright E2E, k6 Load Testing, OWASP, Storybook
9. Azure Deployment Guide
   Azure App Service, Static Web Apps, GitHub Actions CI/CD, Service Principal, Supabase Pooler, Monitoring & Troubleshooting

WealthWise — Project Synopsis

1. Project Title

WealthWise: An Intelligent Portfolio Analytics and Goal-Planning Platform for Indian Mutual Fund Investors

2. Abstract

WealthWise is an enterprise-grade, full-stack web application designed to empower Indian retail mutual fund investors with real-time portfolio intelligence, automated data ingestion, advanced analytics, and goal-based financial planning. The platform bridges the critical gap between raw mutual fund transaction data and actionable investment insights — a space that remains poorly served by existing brokerage platforms and standalone tools in the Indian fintech ecosystem.

At its core, WealthWise ingests mutual fund data through two primary channels: **manual transaction entry** (individual purchases, SIPs, redemptions, switches, SWPs, STPs, and dividends) and **automated CAS (Consolidated Account Statement) PDF parsing**. CAS statements are SEBI-mandated standardized documents issued by CAMS and KFintech that contain the complete transaction history across all folios and AMCs for an investor. WealthWise employs a multi-stage parsing pipeline using Apache PDFBox to extract, normalize, and reconcile scheme-level data with the AMFI master scheme database (containing 45,000+ mutual fund schemes seeded from AMFI's official NAVAll.txt feed).

The analytics engine computes a comprehensive suite of portfolio metrics including:

- **Time-Weighted Returns (TWR)** and **XIRR** (Extended Internal Rate of Return) for both portfolio-wide and per-scheme performance evaluation
- **Annualized Volatility** derived from historical NAV standard deviations
- **Sharpe Ratio** calculations for risk-adjusted return analysis
- **SEBI Riskometer-aligned Risk Profiling** (1–6 scale) with weighted portfolio-level aggregation
- **Fund Overlap Analysis** using stock-level set-intersection logic based on SEBI-mandated category allocation rules (e.g., Large Cap ≥80% from Nifty 100)
- **SIP Intelligence Suite** comprising active SIP detection, monthly outflow analysis, SIP streak tracking, SIP vs. Lumpsum comparison, optimal day-of-month recommendation, and step-up projection modeling
- **Monte Carlo Simulation Engine** (10,000-iteration) for probabilistic goal attainment analysis
- **Deterministic Projection Engine** with sensitivity analysis (return downgrades, missed SIPs, inflation shocks)
- **Required SIP Calculator** with lump-sum bridging and timeline extension modeling

The frontend is built with React 19 and Vite 7, delivering a single-page application (SPA) with a premium dark-mode glassmorphism UI, interactive Recharts-powered data visualizations, and Framer Motion animations. The backend is a stateless REST API built on Spring Boot 3.2.3 with Spring Security (JWT-based authentication), Spring Data JPA, and PostgreSQL (hosted on Supabase). The system is deployed on Render with a multi-stage Docker build for the backend and a static site deployment for the frontend.

Security is a first-class concern: passwords are hashed with BCrypt, PAN card numbers are encrypted at rest with AES-256-GCM (key derived from the JWT secret via SHA-256), JWTs are HMAC-SHA256 signed with configurable expiry, and OTPs for password reset are cryptographically generated using `SecureRandom` and stored in hashed form. The API enforces CORS restrictions, HSTS, X-Frame-Options, X-Content-Type-Options, and XSS protection headers on every response.

3. Problem Statement

> *There exists a critical need for a unified, intelligent, and secure mutual fund portfolio management platform that consolidates investment data from heterogeneous sources, provides institutional-grade analytics (risk profiling, time-weighted returns, fund overlap detection, SIP intelligence), enables goal-based financial planning with probabilistic simulation, and delivers these capabilities through an accessible, modern web interface — all while adhering to Indian regulatory data security requirements for financial applications.*

4. Objectives

The project pursues the following measurable objectives:

ID	OBJECTIVE	SUCCESS METRIC
O1	**Unified Portfolio Consolidation**	Users can view all mutual fund holdings across AMCs/folios in a single dashboard with real-time NAV-based valuations within 3 seconds of page load.
O2	**Automated CAS Ingestion**	The system successfully parses and imports ≥95% of standard CAS PDF formats (CAMS + KFintech) with scheme reconciliation against the AMFI master database.
O3	**Institutional-Grade Analytics**	The analytics engine computes XIRR, TWR, volatility, Sharpe ratio, and risk score with <1% deviation from verified manual calculations.
O4	**Fund Overlap Detection**	The system identifies stock-level overlap between every pair of funds in a user's portfolio using SEBI-mandated category allocation rules.
O5	**SIP Intelligence**	The platform detects active SIPs, tracks streaks, recommends optimal SIP dates, and models step-up scenarios for the next 10+ years.
O6	**Probabilistic Goal Planning**	The Monte Carlo engine runs 10,000 simulations and outputs P10/P50/P90 outcomes with inflation-adjusted values and sensitivity analysis.
O7	**Enterprise-Grade Security**	All authentication uses BCrypt + JWT, PAN cards are AES-256-GCM encrypted at rest, OTPs are hashed and time-limited, and security headers are enforced on every response.
O8	**Production Deployment**	The system is deployable on Render (free tier) with Docker containerization, health checks, and cold-start warmup handling within 90 seconds.

5. Scope

- User registration, authentication (email/password), and session management via JWT
- Password reset via email-based OTP (time-limited, hashed storage)
- User profile management (name, phone, currency, PAN card with AES-256-GCM encryption)
- Manual transaction recording: PURCHASE_LUMPSUM, PURCHASE_SIP, REDEMPTION, SWITCH_IN/OUT, SWP, STP_IN/OUT, DIVIDEND_PAYOUT, DIVIDEND_REINVEST, REVERSAL
- Bulk SIP generator (up to 120 months of monthly transactions in a single request)
- CAS PDF parsing and import with multi-AMC, multi-folio support
- Scheme master database seeded from AMFI NAVAll.txt (45,000+ schemes)
- Real-time NAV fetching from mfapi.in with Caffeine caching (24h for latest, 7d for history)
- Investment lot tracking with FIFO-based redemption unit subtraction and ELSS lock-in enforcement
- Portfolio dashboard with total invested, current value, category allocation, and per-scheme breakdown
- Returns computation: XIRR, TWR, absolute returns, growth timeline
- Risk analytics: volatility, Sharpe ratio, SEBI riskometer scoring, weighted portfolio risk
- Fund overlap analysis using stock-level holdings based on SEBI category rules
- SIP Intelligence Suite: dashboard, SIP vs. lumpsum, day optimization, step-up projection
- Goal creation and management with inflation-adjusted targets

- Goal analysis: Monte Carlo simulation, deterministic projection with sensitivity, required SIP calculator
- Goal-to-fund linking with allocation percentages
- Responsive dark-mode UI with glassmorphism design
- Deployment on Render (Docker for backend, static site for frontend)

6. Technology Stack

6.1 Frontend

TECHNOLOGY	VERSION	JUSTIFICATION
React	19.2.0	Component-based architecture enabling reusable UI elements; concurrent rendering for smooth transitions; largest ecosystem of tooling and community support.
Vite	7.3.1	Near-instant HMR (Hot Module Replacement) during development; ESBuild-powered bundling delivers 10–100× faster builds than webpack; native ES module support.
React Router DOM	7.14.0	Declarative client-side routing for SPA navigation; supports protected route wrappers, nested routes, and catch-all redirects.
Recharts	3.8.0	React-native charting library built on D3; supports responsive containers, custom tooltips, and animated transitions for line charts, pie charts, bar charts, and area charts.
Framer Motion	12.35.2	Production-grade animation library for React; used for page transitions, modal animations, card hover effects, and scroll-triggered reveals.
Lucide React	0.577.0	Lightweight, tree-shakeable icon library; consistent design language across 1,500+ icons.
Supabase Client	2.103.0	Client-side Supabase SDK used for optional direct database queries for auxiliary features (e.g., real-time market data display).
Inter + Space Grotesk	Google Fonts	Modern, highly legible typefaces optimized for data-dense financial interfaces; Inter for body text, Space Grotesk for headings and numerical displays.

6.2 Backend

TECHNOLOGY	VERSION	JUSTIFICATION
Java	17 (LTS)	Long-term support release with modern language features (records, sealed classes, pattern matching); required by Spring Boot 3.x.
Spring Boot	3.2.3	Convention-over-configuration framework that provides auto-configured web server, dependency injection, and production-ready feature defaults.
Spring Security	6.x	Industry-standard security framework; configured for stateless JWT authentication with custom `OncePerRequestFilter`.
Spring Data JPA	3.x	Abstraction over Hibernate ORM; provides repository pattern with automatic query generation and support for native SQL queries.
Spring Boot Starter Mail	3.x	JavaMail abstraction for sending OTP emails via Gmail SMTP; supports STARTTLS and app-password authentication.
Spring Boot Starter Validation	3.x	Bean Validation 3.0 (Jakarta) for request body validation with `@Valid`, `@Positive`, `@Min`, `@Max`, `@DecimalMin`, `@DecimalMax`.
Spring Boot Starter Cache + Caffeine	3.x / 3.1.8	In-process caching (like Redis but without infrastructure overhead); three caches: `nav_latest` (24h TTL, 50K entries), `nav_history` (7d TTL, 500K entries), `scheme_meta` (1h TTL, 10K entries).
JJWT	0.12.5	Modern JWT library for creating/validating HMAC-SHA256 signed tokens; supports configurable expiry.
Apache PDFBox	2.0.28	Pure Java PDF parsing library for extracting text from CAS PDF statements; no OS-level dependencies.

TECHNOLOGY	VERSION	JUSTIFICATION
Lombok	Latest	Compile-time code generation for `@RequiredArgsConstructor`, `@Data`; reduces boilerplate in service and DTO classes.
BCryptPasswordEncoder	Spring Security	Adaptive one-way hashing with configurable cost factor; industry standard for password storage.

6.3 Database

TECHNOLOGY	JUSTIFICATION
PostgreSQL 15 (via Supabase)	ACID-compliant relational database; supports `INSERT ON CONFLICT` for atomic upserts on NAV history; rich indexing (B-tree, partial); JSON support for future extensibility; Supabase provides managed hosting with automated backups and connection pooling.

6.4 Deployment & DevOps

TECHNOLOGY	JUSTIFICATION
Docker (Multi-stage)	Stage 1: Maven 3.8.7 + Temurin JDK 17 for compilation; Stage 2: Temurin JRE 17 (alpine-based) for runtime. Results in a minimal, secure image (~200 MB).
Render	Platform-as-a-Service supporting Docker web services (backend) and static sites (frontend); free tier suitable for demonstration.
JVM Tuning	`-XX:TieredStopAtLevel=1` (skip C2 JIT for faster startup), `-XX:MaxRAMPercentage=70.0`, `-XX:+UseSerialGC` (lower memory footprint), `-Xss512k` (reduced per-thread stack).

6.5 External APIs

API	PURPOSE
mfapi.in	Open-source Indian mutual fund API providing latest and historical NAV data for all AMFI-registered schemes; used for real-time valuation and returns computation.
AMFI NAVAll.txt	Official daily NAV file published by the Association of Mutual Funds in India; parsed to seed and update the scheme master database with 45,000+ schemes including categorization, AMC, and plan type data.

WealthWise — Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive, detailed description of the functional and non-functional requirements for the WealthWise portfolio analytics platform. It serves as the binding contract between the development team and stakeholders, defining what the system shall do, under what constraints, and to what quality standards.

This document is intended for:

- **Development team** — as the authoritative reference for implementation
- **Academic evaluators** — as evidence of rigorous requirements engineering
- **Quality assurance** — as the basis for test case derivation
- **Future maintainers** — as the canonical specification for feature evolution

1.2 Scope

WealthWise is a web-based mutual fund portfolio management and analytics platform targeting Indian retail investors. The system encompasses:

- **Authentication & Identity Management** — Secure user registration, login, password recovery, and profile management
- **Transaction Management** — Manual entry, bulk SIP generation, CAS PDF import, and transaction reversal
- **Portfolio Intelligence** — Real-time NAV-based valuation, returns computation (XIRR, TWR), risk analytics, and fund overlap detection
- **SIP Intelligence** — Active SIP tracking, streak analysis, SIP vs. lumpsum comparison, day optimization, and step-up projections
- **Goal-Based Planning** — Goal creation with inflation-adjusted targets, Monte Carlo simulation, deterministic projections, and required SIP calculations
- **Scheme Data Management** — AMFI master database seeding, scheme search, and NAV caching

The system does **not** include: brokerage API integrations, payment processing, tax computation, mobile native apps, or multi-currency FX conversion.

1.3 Definitions, Acronyms, and Abbreviations

TERM	DEFINITION
AMFI	Association of Mutual Funds in India — the industry body that assigns unique scheme codes
NAV	Net Asset Value — the per-unit price of a mutual fund scheme, published daily
CAS	Consolidated Account Statement — a SEBI-mandated document listing all MF transactions across AMCs
CAMS	Computer Age Management Services — one of two primary MF registrars in India
KFintech	Formerly Karvy — the second primary MF registrar in India
SIP	Systematic Investment Plan — a recurring monthly investment in a mutual fund
SWP	Systematic Withdrawal Plan — a recurring monthly withdrawal from a mutual fund
STP	Systematic Transfer Plan — a recurring transfer from one fund to another
XIRR	Extended Internal Rate of Return — IRR for irregular cash flows
TWR	Time-Weighted Return — return metric that eliminates the impact of cash flow timing

TERM	DEFINITION
ELSS	Equity Linked Savings Scheme — tax-saving MF with 3-year lock-in under Section 80C
SEBI	Securities and Exchange Board of India — the financial markets regulator
PAN	Permanent Account Number — 10-character alphanumeric tax identifier issued by the IT Department
AES-256-GCM	Advanced Encryption Standard with 256-bit key and Galois/Counter Mode (authenticated encryption)
JWT	JSON Web Token — a compact, URL-safe means of representing claims between two parties
BCrypt	An adaptive password hashing function based on the Blowfish cipher
FIFO	First In, First Out — the method used for matching redemption units against purchase lots
OTP	One-Time Password — a time-limited code used for identity verification

2. Overall Description

2.1 Product Perspective

WealthWise operates as a **standalone web application** that does not depend on or integrate with any proprietary brokerage platform. It is designed as a post-trade analytics tool — users import existing transaction data (via CAS PDF or manual entry), and the system enriches this data with real-time NAV feeds, scheme metadata, and computed analytics.

The system follows a **three-tier architecture**:

1. **Presentation Tier** — React 19 SPA served as static files from a CDN (Render static site)
2. **Application Tier** — Spring Boot 3.2 REST API deployed as a Docker container on Render
3. **Data Tier** — PostgreSQL 15 hosted on Supabase with connection pooling

External dependencies are limited to two read-only public APIs:

- **mfapi.in** — NAV data retrieval (no API key required)
- **AMFI NAVAll.txt** — Scheme master data seeding (public HTTP endpoint)

2.2 Product Functions (Summary)

WealthWise Product Functions Mindmap

2.3 User Classes and Characteristics

USER CLASS	DESCRIPTION	TECHNICAL PROFICIENCY	USAGE PATTERN
Retail Investor	Individual investing in mutual funds through SIPs or lump sums. Holds 2–15 funds across 1–5 AMCs.	Low to Medium	Weekly dashboard checks; monthly transaction entry; quarterly analytics review
Active Trader	High-frequency investor who regularly rebalances portfolio, switches between funds, and monitors performance daily.	Medium to High	Daily dashboard access; frequent transaction entry; heavy analytics usage
Financial Planner	Uses the platform to analyze client portfolios (via CAS imports) and demonstrate goal feasibility.	High	CAS imports for clients; goal analysis sessions; risk profiling
System Administrator	Manages scheme master data, triggers NAV seeding, and monitors system health.	High	Periodic scheme seeding; reconciliation of synthetic codes; health monitoring

3. Functional Requirements

3.1 Module M01 — Authentication & User Management

FR-01: User Registration

- **ID:** FR-01
- **Priority:** High
- **Description:** The system shall allow new users to register by providing full name, email, password, phone number (optional), currency preference (optional), and PAN card number (optional).
- **Input:** `{ fullName, email, password, phone?, currency?, panCard? }`
- **Processing:**
  1. Validate that the email is not already registered (unique constraint on `users.email`)
  2. Hash the password using BCrypt with default cost factor (10 rounds)
  3. If PAN card is provided, encrypt using AES-256-GCM before database storage
  4. Generate a JWT token with the user's email as the subject and a configurable expiry
  5. Return the JWT token and sanitized user object (password field set to null)
- **Output:** `{ token: string, user: { id, fullName, email, ... } }`
- **Error Cases:**
  - Email already registered → 400 with `{ error: "Email is already registered!" }`
  - Invalid input → 400 with descriptive error message

FR-02: User Authentication (Sign In)

- **ID:** FR-02
- **Priority:** High
- **Description:** The system shall authenticate users via email and password, returning a JWT on success.
- **Input:** `{ email, password }`
- **Processing:**
  1. Look up user by email
  2. Compare provided password against BCrypt hash using `PasswordEncoder.matches()`
  3. On success, generate JWT token
  4. Return token and sanitized user object
- **Output:** `{ token: string, user: { id, fullName, email, ... } }`
- **Error Cases:**
  - Invalid credentials → 401 with `{ error: "Invalid email or password" }`

FR-03: Password Recovery (Forgot Password)

- **ID:** FR-03
- **Priority:** High
- **Description:** The system shall support password recovery via email-based OTP verification.
- **Sub-Flow 1: Generate OTP**
  - Input: `{ email }`
  - Processing: Generate 6-digit OTP using `SecureRandom`; hash OTP with BCrypt; store hash + 5-minute expiry in user record; send plaintext OTP to email via SMTP
  - Output: `{ message: "OTP generated and sent to email successfully." }`
- **Sub-Flow 2: Verify OTP and Reset Password**
  - Input: `{ email, otp, newPassword }`
  - Processing: Match OTP against stored hash; verify expiry; hash new password; clear OTP fields
  - Output: `{ message: "Password has been successfully reset." }`
- **Error Cases:**

- Email not found → 400
- Invalid or expired OTP → 400
- OTP expiry check: `otpExpiry.isBefore(LocalDateTime.now())`

FR-04: User Profile Management

- **ID:** FR-04
- **Priority:** Medium
- **Description:** Authenticated users shall be able to view and update their profile, and change their password.
- **FR-04.1: Get Profile** — Returns user object with PAN masked (format: `ABCDE****F`)
- **FR-04.2: Update Profile** — Updates fullName, phone, currency, panCard fields
- **FR-04.3: Change Password** — Requires current password verification; new password must be ≥8 characters

FR-05: Health Check Endpoint

- **ID:** FR-05
- **Priority:** High
- **Description:** The system shall expose a public health check endpoint at `GET /api/auth/health` that returns `{ status: "UP" }` for use by Render's health check polling and the frontend warmup mechanism.

3.2 Module M02 — Scheme Master Management

FR-06: Scheme Database Seeding

- **ID:** FR-06
- **Priority:** High
- **Description:** The system shall seed the scheme master database by parsing AMFI's NAVAll.txt file, extracting scheme metadata (AMFI code, ISIN, scheme name, AMC, plan type, option type, fund type, SEBI category, broad category, risk level, and latest NAV).
- **Processing:**
  1. Fetch NAVAll.txt from `https://www.amfiindia.com/spages/NAVAll.txt`
  2. Parse pipe-delimited fields: `Scheme Code;ISIN Div Payout/ISIN Growth;ISIN Div Reinvestment;Scheme Name;Net Asset Value;Date`
  3. Infer `planType` (DIRECT/REGULAR), `optionType` (GROWTH/IDCW), `fundType` (OPEN_ENDED/CLOSE_ENDED) from scheme name
  4. Map SEBI category from section headers in the file
  5. Derive `broadCategory` (EQUITY/DEBT/HYBRID/SOLUTION/OTHER) and `riskLevel` (1–6) from SEBI category
  6. Upsert into `scheme_master` table (insert or update on `amfi_code` conflict)
- **Output:** `{ totalProcessed, newInserted, updated }`

FR-07: Scheme Search

- **ID:** FR-07
- **Priority:** High
- **Description:** The system shall provide a paginated, filterable search over the scheme master database.
- **Parameters:** `q` (search term), `category` (e.g., EQUITY), `planType` (e.g., DIRECT), `page`, `size`
- **Output:** Paginated result with `{ content, totalElements, totalPages, page }`

FR-08: Scheme Lookup by AMFI Code

- **ID:** FR-08
- **Priority:** Medium
- **Description:** The system shall return scheme metadata for a given AMFI code. Returns 404 if not found.

3.3 Module M03 — NAV Management

FR-09: Latest NAV Retrieval

- **ID:** FR-09
- **Priority:** High
- **Description:** The system shall fetch the latest NAV for a given AMFI code from mfapi.in, with a 24-hour Caffeine cache.
- **Cache Strategy:** First check `nav_latest` cache → if miss, fetch from `https://api.mfapi.in/mf/{amfiCode}/latest` → cache result for 24h

FR-10: Historical NAV Retrieval

- **ID:** FR-10
- **Priority:** High
- **Description:** The system shall fetch and persist the full historical NAV data for a scheme, with a 7-day Caffeine cache.
- **Processing:** Fetch from mfapi.in → persist each NAV entry to `nav_history` table using atomic `INSERT ON CONFLICT DO NOTHING` → cache in memory for 7 days

FR-11: NAV for Specific Date

- **ID:** FR-11
- **Priority:** Medium
- **Description:** The system shall return the NAV for a scheme on a specific date. If no NAV exists for that date (market holiday), the system shall return null with an advisory message to use the nearest available NAV.

FR-12: NAV Cache Refresh

- **ID:** FR-12
- **Priority:** Medium
- **Description:** The system shall support forced NAV refresh via `POST /api/nav/refresh/{amfiCode}`, bypassing the cache and fetching fresh data from mfapi.in.

3.4 Module M04 — Transaction Management

FR-13: Record Transaction

- **ID:** FR-13
- **Priority:** High
- **Description:** The system shall record a mutual fund transaction with automatic investment lot creation and management.
- **Supported Types:** PURCHASE_LUMPSUM, PURCHASE_SIP, REDEMPTION, SWITCH_IN, SWITCH_OUT, SWP, STP_IN, STP_OUT, DIVIDEND_PAYOUT, DIVIDEND_REINVEST
- **Processing for Purchase Types:**
  1. Generate unique transaction reference (`TXN-{userId}-{timestamp}-{random}`)
  2. If NAV not provided, fetch latest NAV from mfapi.in
  3. Compute units = amount / NAV (scale: 6 decimals)
  4. Apply 0.005% stamp duty for purchases
  5. Create `InvestmentLot` record with `unitsOriginal = unitsRemaining = computed units`
  6. If ELSS scheme, set `isElss = true` and `elssLockUntil = purchaseDate + 3 years`
- **Processing for Redemption Types:**
  1. Fetch active investment lots for the scheme (ordered by purchase date ASC — FIFO)
  2. Subtract redeemed units from lots, zeroing out fully redeemed lots
  3. Validate sufficient units available; throw `IllegalStateException` if shortfall
- **Edge Cases:**

- Partial lot redemption (lot has more units than redeemed)
- Multi-lot redemption spanning 3+ lots
- ELSS lock-in violation (redemption before 3-year lock)

FR-14: Bulk SIP Generator

- **ID:** FR-14
- **Priority:** Medium
- **Description:** The system shall generate multiple SIP transactions at once for a specified date range (up to 120 months / 10 years).
- **Input:** `{ schemeAmfiCode, schemeName, folioNumber, amount, startDate, endDate }`
- **Processing:** For each month in the range, create a PURCHASE_SIP transaction on the SIP date (1st of each month), fetching historical NAV for each date.

FR-15: CAS PDF Import

- **ID:** FR-15
- **Priority:** High
- **Description:** The system shall parse an uploaded CAS PDF file and import all transactions.
- **Processing:**
  1. Validate file is PDF (content-type check)
  2. Extract text from all pages using Apache PDFBox
  3. Parse folio numbers, scheme names, and transaction rows using regex patterns
  4. Map scheme names to AMFI codes using the scheme master database
  5. For unresolvable schemes, generate synthetic `WW_ISIN_*` codes and flag for later reconciliation
  6. Create transactions and investment lots for all parsed entries
  7. Log the upload in `cas_upload_log` with status (SUCCESS/FAILED), total folios, and total transactions
- **Output:** `{ totalFolios, totalTransactions, syntheticCodes[], status }`

FR-16: Synthetic Scheme Reconciliation

- **ID:** FR-16
- **Priority:** Medium
- **Description:** The system shall reconcile synthetic `WW_ISIN_*` codes generated during CAS import with real AMFI codes by searching mfapi.in. The operation is idempotent and atomically updates transactions, investment lots, and scheme master records.

FR-17: Transaction Reversal

- **ID:** FR-17
- **Priority:** Medium
- **Description:** The system shall support reversing a transaction by creating a mirror transaction of opposite type. Reversal of a PURCHASE creates a REDEMPTION (returning units to the lot), and vice versa.

FR-18: Portfolio Summary

- **ID:** FR-18
- **Priority:** High
- **Description:** The system shall compute and return a per-scheme portfolio summary including: scheme name, total units held, total invested amount, current NAV, current value, unrealized P&L, and percentage return.

3.5 Module M05 — Returns & Analytics

FR-19: Portfolio Returns

- **ID:** FR-19
- **Priority:** High

- **Description:** The system shall compute portfolio-level returns including total invested, current value, XIRR, absolute return, and a growth timeline (monthly portfolio value snapshots for charting).

FR-20: Per-Scheme Returns

- **ID:** FR-20
- **Priority:** Medium
- **Description:** The system shall compute returns for a specific scheme including invested amount, current value, XIRR, TWR, and a scheme-level growth timeline.

FR-21: Risk Profiling

- **ID:** FR-21
- **Priority:** High
- **Description:** The system shall compute portfolio risk metrics:
  - **Annualized Volatility** — Standard deviation of daily NAV returns, annualized (×√252)
  - **Sharpe Ratio** — (Portfolio Return − Risk-Free Rate) / Volatility
  - **Weighted Risk Score** — Portfolio-level SEBI riskometer score (1–6) weighted by allocation
  - **Risk Category** — CONSERVATIVE / MODERATE / AGGRESSIVE based on weighted score thresholds

FR-22: Fund Overlap Analysis

- **ID:** FR-22
- **Priority:** Medium
- **Description:** The system shall compute pairwise stock-level overlap between all funds in a user's portfolio.
- **Processing:**
  1. For each scheme, retrieve stock holdings from `fund_holdings` table (populated by `FundHoldingsIngestionService` based on SEBI category rules)
  2. Compute Jaccard overlap: `|A ∩ B| / |A ∪ B|` for every pair of schemes
  3. Return overlap matrix, high-overlap pairs (>30%), and consolidation suggestions

FR-23: User Risk Profile Save

- **ID:** FR-23
- **Priority:** Low
- **Description:** The system shall allow users to manually set their risk profile (CONSERVATIVE / MODERATE / AGGRESSIVE) which is persisted in the `users.risk_profile` column.

3.6 Module M06 — SIP Intelligence

FR-24: SIP Dashboard

- **ID:** FR-24
- **Priority:** Medium
- **Description:** The system shall analyze transaction history to detect active SIPs and compute: total active SIPs, monthly outflow, next step-up date, projected corpus, SIP streak (consecutive months maintained), and health alerts.
- **SIP Detection Logic:** A SIP is considered active if there are ≥3 PURCHASE_SIP transactions for the same scheme with consistent monthly intervals in the last 6 months.

FR-25: SIP vs. Lumpsum Comparison

- **ID:** FR-25
- **Priority:** Medium
- **Description:** The system shall compare SIP returns against a hypothetical lumpsum investment of the same total amount invested on the SIP start date.

FR-26: SIP Day Optimization

- **ID:** FR-26

- **Priority:** Low
- **Description:** The system shall recommend the optimal day of the month for SIP execution based on historical NAV volatility patterns.

FR-27: SIP Step-Up Projection

- **ID:** FR-27
- **Priority:** Medium
- **Description:** The system shall project SIP outcomes with and without annual step-up (default: 10% per year) over the remaining investment horizon.

3.7 Module M07 — Goal Planning

FR-28: Goal CRUD Operations

- **ID:** FR-28
- **Priority:** Medium
- **Description:** Users shall be able to create, read, update, and delete financial goals with attributes: goal name, type (RETIREMENT, HOUSE, EDUCATION, WEDDING, EMERGENCY, VACATION, CAR, CUSTOM), icon, target amount (today's money), inflation rate, target date, priority (LOW/MEDIUM/HIGH), monthly SIP allocation, and expected return rate.

FR-29: Goal Analysis Engine

- **ID:** FR-29
- **Priority:** High
- **Description:** The system shall provide a unified analysis endpoint that runs three engines simultaneously:
- **FR-29.1: Monte Carlo Simulation** — 10,000 iterations with Gaussian random returns; floor limiting by volatility tier; output: P10/P50/P90 in today's money; probability of success against inflation-adjusted target
- **FR-29.2: Deterministic Projection** — Simple compound growth projection with sensitivity analysis: (a) Return = 10% pa scenario, (b) 6 SIPs missed scenario, (c) Inflation +2% scenario; output: projected corpus, gap, on-track boolean
- **FR-29.3: Required SIP Calculator** — Computes the required monthly SIP to meet the goal, current SIP gap, equivalent lump-sum today, and additional months needed at current SIP rate

FR-30: Goal-Fund Linking

- **ID:** FR-30
- **Priority:** Low
- **Description:** Users shall be able to link investment lots to goals with allocation percentages (1–100%), enabling tracking of how much of each goal's target is funded by existing investments.

---

4. Non-Functional Requirements

4.1 Performance

ID	REQUIREMENT	TARGET
NFR-01	Dashboard page load time (after backend warm)	≤ 3 seconds
NFR-02	Analytics computation (risk + overlap + SIP)	≤ 5 seconds for portfolios with ≤50 schemes
NFR-03	CAS PDF parsing (10-page document)	≤ 10 seconds
NFR-04	Monte Carlo simulation (10,000 iterations)	≤ 2 seconds
NFR-05	Scheme search query response time	≤ 500 ms
NFR-06	Backend cold-start time (Docker on Render free tier)	≤ 90 seconds

4.2 Scalability

ID	REQUIREMENT
NFR-07	Support up to 10,000 registered users with independent portfolios
NFR-08	Scheme master database supports 50,000+ schemes without performance degradation
NFR-09	NAV history table supports 10M+ rows with indexed queries <200ms
NFR-10	Caffeine cache sized for 50K latest NAV entries and 500K historical entries

4.3 Security

ID	REQUIREMENT
NFR-11	Passwords hashed with BCrypt (cost factor 10, adaptive)
NFR-12	PAN card numbers encrypted at rest with AES-256-GCM
NFR-13	JWT tokens signed with HMAC-SHA256 using a ≥32-character secret
NFR-14	OTPs generated with `SecureRandom` (cryptographic PRNG), stored hashed, with 5-minute expiry
NFR-15	CORS restricted to configured origins only
NFR-16	Security headers on every response: X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, HSTS, Referrer-Policy
NFR-17	API error messages sanitized — no Java class paths or stack traces exposed to clients
NFR-18	Sensitive configuration (DB password, JWT secret, SMTP password) externalized via environment variables; never committed to version control

4.4 Reliability

ID	REQUIREMENT
NFR-19	NAV data persisted to database as write-through cache; survives server restarts
NFR-20	Unique constraint on `(amfi_code, nav_date)` in `nav_history` prevents duplicate entries
NFR-21	CAS upload logged with status audit trail; errors captured in `cas_upload_log.error_message`
NFR-22	Global exception handler prevents unhandled exceptions from leaking internal state
NFR-23	Transaction reversal maintains referential integrity via `reversal_of` foreign key
NFR-24	Backend warmup mechanism allows UI to function within 90 seconds of cold start

4.5 Usability

ID	REQUIREMENT
NFR-25	Dark mode default with premium glassmorphism aesthetic
NFR-26	Responsive layout supporting desktop (1920px), laptop (1366px), and tablet (768px) viewports
NFR-27	Form validation with real-time error messages
NFR-28	Loading states with animated spinners for all asynchronous operations
NFR-29	Error boundary component prevents full-page crashes; displays user-friendly fallback

ID	REQUIREMENT
NFR-30	Backend warmup overlay with elapsed time display and animated loading state

5. System Features (Summary Matrix)

FEATURE ID	FEATURE NAME	MODULE	PRIORITY	COMPLEXITY
F01	User Sign Up	M01	High	Low
F02	User Sign In	M01	High	Low
F03	Forgot Password (OTP)	M01	High	Medium
F04	Profile Management	M01	Medium	Low
F05	PAN Card Encryption	M01	High	Medium
F06	AMFI Scheme Seeding	M02	High	High
F07	Scheme Search	M02	High	Low
F08	Latest NAV Fetch	M03	High	Medium
F09	Historical NAV Fetch	M03	High	Medium
F10	Manual Transaction Entry	M04	High	High
F11	Bulk SIP Generator	M04	Medium	Medium
F12	CAS PDF Import	M04	High	Very High
F13	Transaction Reversal	M04	Medium	Medium
F14	Portfolio Summary	M04	High	Medium
F15	XIRR & TWR Computation	M05	High	High
F16	Risk Profiling	M05	High	High
F17	Fund Overlap Analysis	M05	Medium	Very High
F18	SIP Dashboard	M06	Medium	High
F19	SIP vs. Lumpsum	M06	Medium	Medium
F20	SIP Day Optimization	M06	Low	Medium
F21	SIP Step-Up Projection	M06	Medium	Medium
F22	Goal CRUD	M07	Medium	Low
F23	Monte Carlo Simulation	M07	High	High
F24	Deterministic Projection	M07	High	Medium
F25	Required SIP Calculator	M07	Medium	Medium
F26	Goal-Fund Linking	M07	Low	Medium

6. Use Case Specifications

UC-01: User Registers and Creates First Transaction

FIELD	DETAIL
Use Case ID	UC-01
Title	New User Onboarding and First Transaction
Primary Actor	Retail Investor
Preconditions	User has access to a web browser and a valid email address
Trigger	User clicks "Get Started" on the landing page

Main Flow:

1. User clicks "Get Started" → Auth modal opens on "Sign Up" tab
2. User enters full name, email, password → clicks "Create Account"
3. System validates email uniqueness, hashes password, creates user → returns JWT
4. User is redirected to Dashboard (protected route)
5. Dashboard shows empty state with "Record your first transaction" prompt
6. User navigates to Transactions page → clicks "Add Transaction"
7. User selects scheme (live search with debounce), enters amount, selects date
8. System auto-fetches NAV for the selected date from mfapi.in
9. System computes units (amount / NAV), applies stamp duty, creates transaction + investment lot
10. Dashboard now shows the investment with current value based on latest NAV

Alternate Flows:

- **2a.** Email already registered → Error message displayed; suggest "Sign In" instead
- **8a.** NAV not available for the selected date (holiday) → User prompted to select nearest business day
- **9a.** Scheme is ELSS → System sets `is_elss = true` and `elss_lock_until = date + 3 years`

Edge Cases:

- User enters PAN card during registration → PAN is AES-256-GCM encrypted before database storage
- User's JWT expires during the transaction entry → ProtectedRoute detects expiry, redirects to landing page

---

UC-02: Import CAS Statement and View Portfolio

FIELD	DETAIL
Use Case ID	UC-02
Title	CAS PDF Import and Portfolio Consolidation
Primary Actor	Retail Investor
Preconditions	User is authenticated; has a CAS PDF from CAMS/KFintech
Trigger	User clicks "Import CAS" on the Transactions page

Main Flow:

1. User clicks "Import CAS" → file picker opens
2. User selects CAS PDF → system validates file type (application/pdf)
3. System extracts text via PDFBox → parses folio numbers, scheme names, transaction rows
4. For each scheme, system searches `scheme_master` by name → maps to AMFI code
5. For unresolvable schemes, system generates synthetic `WW_ISIN_*` code
6. System creates transactions and investment lots for all parsed entries

7. System logs upload in `cas_upload_log` → returns summary
8. User sees imported transactions on the Transactions page
9. Dashboard updates with consolidated portfolio view

Alternate Flows:

- **2a.** File is not PDF → Error: "Valid PDF file is required"
- **4a.** Scheme name partially matches multiple entries → Best match selected by Levenshtein distance
- **5a.** Synthetic codes generated → Banner displayed suggesting "Run Reconciliation"

UC-03: Analyze Portfolio Risk and View Fund Overlap

FIELD	DETAIL
Use Case ID	UC-03
Title	Portfolio Analytics with Risk and Overlap
Primary Actor	Active Trader
Preconditions	User has ≥2 schemes with transactions in their portfolio
Trigger	User navigates to the Analytics page

Main Flow:

1. Analytics page loads → fires three parallel API calls: `/api/analytics/risk`, `/api/analytics/overlap`, `/api/sip/dashboard`
2. Risk profile section displays: portfolio volatility, Sharpe ratio, SEBI risk score, risk category badge
3. Fund overlap section displays: pairwise overlap matrix (heatmap), high-overlap pairs (>30%), and consolidation suggestions
4. SIP intelligence section displays: active SIPs count, monthly outflow, streak, projected corpus
5. User can click on individual fund pairs to see overlapping stock names

Edge Cases:

- Portfolio has only 1 scheme → Overlap matrix shows "Add more funds to see overlap"
- NAV history insufficient (<30 days) → Volatility shows "Insufficient data" warning
- All schemes are synthetic `WW_ISIN_*` → Analytics skips NAV-dependent calculations

---

UC-04: Create a Goal and Run Monte Carlo Analysis

FIELD	DETAIL
Use Case ID	UC-04
Title	Goal Creation with Probabilistic Feasibility Analysis
Primary Actor	Financial Planner
Preconditions	User is authenticated
Trigger	User navigates to Goals page and clicks "Create New Goal"

Main Flow:

1. User fills goal wizard: name ("Retirement Fund"), type (RETIREMENT), target amount (₹5,00,00,000), inflation rate (6%), target date (2050-01-01), monthly SIP (₹25,000), expected return rate (12% pa)
2. System computes `target_amount_future` (inflation-adjusted) and `years_remaining`
3. User clicks "Analyze" → frontend sends POST to `/api/learn/analyse`
4. Backend runs three engines simultaneously:
    - Monte Carlo: 10,000 simulations → P10 = ₹2.1Cr, P50 = ₹4.8Cr, P90 = ₹9.2Cr, Probability = 42.5%

    - Deterministic: Projected = ₹4.6Cr, Gap = ₹40L, NOT on track
    - Required SIP: Need ₹32,000/month (gap: ₹7,000), or ₹18L lump sum today
5. User sees results in an interactive modal with scenario sensitivity analysis

Edge Cases:

- Target already achieved (gap ≤ 0) → "On Track" badge displayed; required SIP = 0
- Monthly contribution = 0 → Monte Carlo runs with corpus-only growth
- Extreme volatility (stdDev > 0.07) → Floor limit set to -30% per month to prevent mathematical explosion

WealthWise — System Design Document (HLD + LLD)

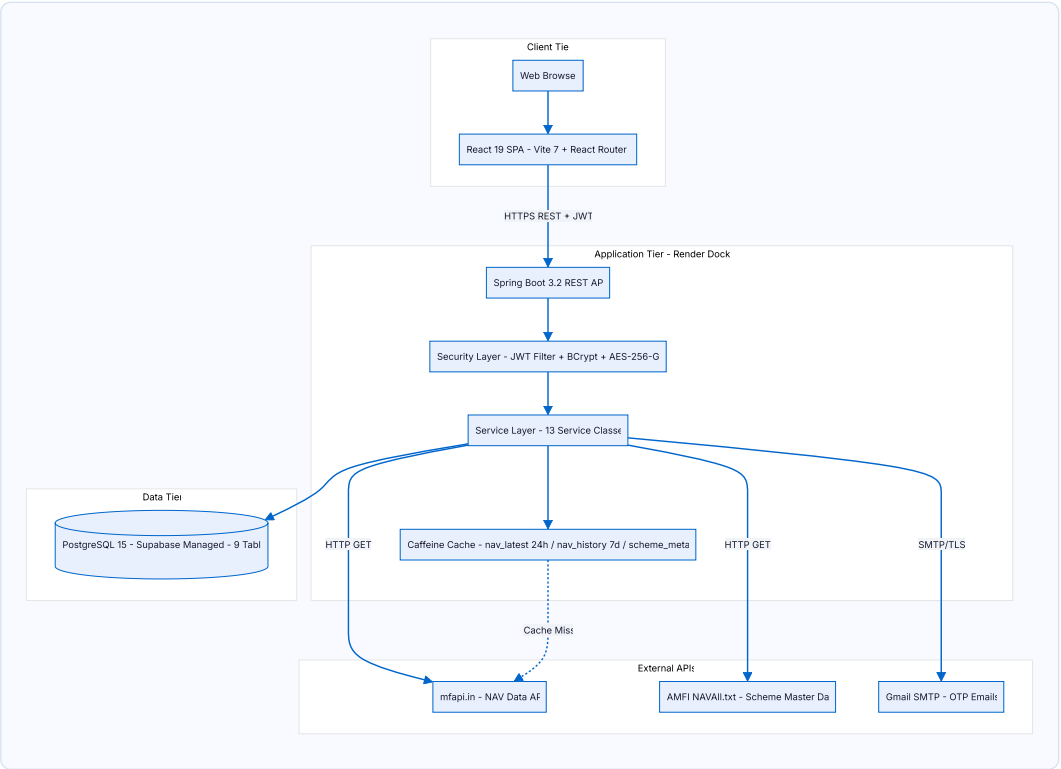
PART I: HIGH-LEVEL DESIGN (HLD)

1. Architecture Overview

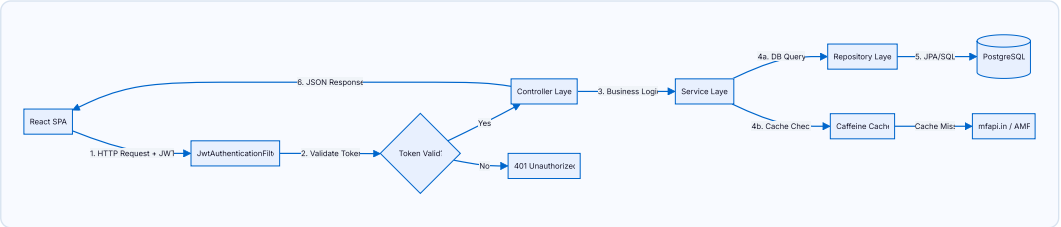
WealthWise follows a **three-tier client-server architecture** with strict separation of concerns. The design prioritizes:

1. **Statelessness** — No server-side sessions; all authentication state carried in JWT tokens
2. **API-First Design** — Backend exposes a pure REST API; frontend is a completely decoupled SPA
3. **Cache-Layered Data Access** — Caffeine in-memory caching reduces external API calls by 95%+
4. **Security-by-Default** — Every response includes hardened security headers; sensitive data encrypted at rest

1.1 Architecture Diagram



1.2 Component Interaction Flow



1.3 Request Processing Pipeline

Every API request passes through the following pipeline:

STAGE	COMPONENT	RESPONSIBILITY
1	`WebConfig` CORS Interceptor	Validate origin against allow-list; set security headers (HSTS, X-Frame-Options, etc.)
2	`JwtAuthenticationFilter`	Extract `Authorization: Bearer <token>` header; validate JWT signature + expiry; resolve email → userId; set `request.setAttribute("userId", ...)`
3	Controller	Parse request body/params; delegate to service; wrap result in `ResponseEntity`
4	Service	Execute business logic; interact with repositories and external APIs; compute analytics
5	Repository	JPA-managed database interactions; Spring Data auto-generated queries + native SQL
6	`GlobalExceptionHandler`	Catch unhandled exceptions; return sanitized error JSON (no stack traces)

---

2. Technology Justification

2.1 Why Spring Boot over Node.js/Express?

CRITERION	SPRING BOOT	NODE.JS/EXPRESS
Type Safety	Java 17 with compile-time type checking prevents runtime type errors in financial calculations	JavaScript's dynamic typing increases risk of subtle numerical bugs
Financial Precision	Native `BigDecimal` support with configurable precision (18,4 for amounts, 18,6 for units)	JavaScript `Number` uses IEEE 754 doubles — inherent floating-point precision loss
Thread Model	Multi-threaded; Monte Carlo 10K iterations utilize thread pool efficiently	Single-threaded event loop; CPU-bound Monte Carlo blocks the entire server
Security Framework	Spring Security provides enterprise-grade filter chain, BCrypt, CORS, CSRF	Manual middleware assembly with limited security abstractions
ORM Maturity	JPA/Hibernate — 20+ years of production hardening; relationship mapping, lazy loading, L2 cache	Sequelize/TypeORM — less mature; weaker migration tooling
PDF Parsing	Apache PDFBox — pure Java, no native dependencies, battle-tested	pdf.js works but slower; native alternatives (pdftotext) add system dependencies

2.2 Why PostgreSQL over MySQL/MongoDB?

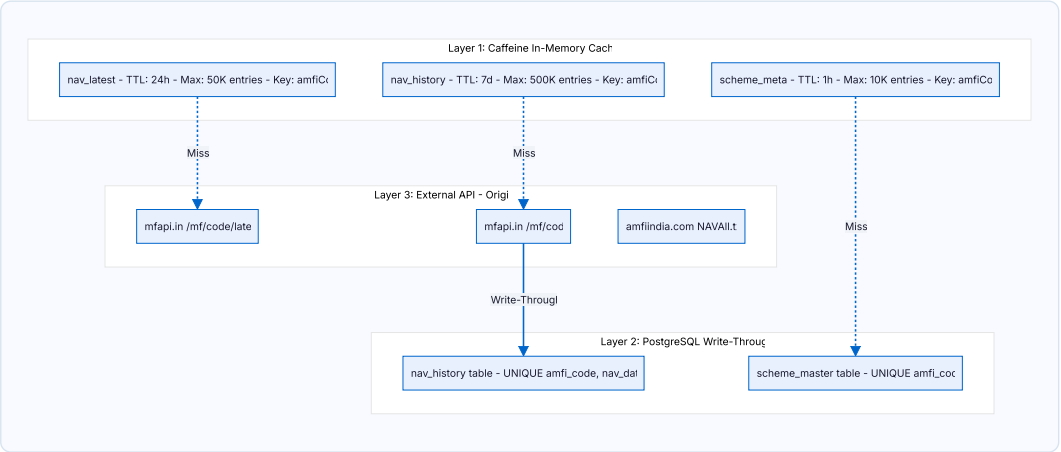
CRITERION	POSTGRESQL
ACID Compliance	Full transactional support for financial data; critical for concurrent lot modifications
INSERT ON CONFLICT	Native upsert syntax used for idempotent NAV history ingestion (prevents duplicate violations)
Window Functions	Used in analytics queries for running totals, rank-based lot selection
JSON Support	`jsonb` type available for future extensibility (e.g., storing raw CAS parse results)
Supabase Ecosystem	Managed hosting with auto-backups, connection pooling, Row Level Security, and a web dashboard

2.3 Why React 19 over Angular/Vue?

CRITERION	REACT 19
Concurrent Rendering	React 19's `useTransition` enables smooth analytics chart rendering without blocking user input
Ecosystem	Recharts, Framer Motion, React Router — all first-class React integrations

CRITERION	REACT 19
Component Model	Functional components with hooks enable clean separation of data fetching (AuthContext), state management (useState/useEffect), and presentation
Community & Hiring	Largest frontend ecosystem; most tutorials, libraries, and developer availability

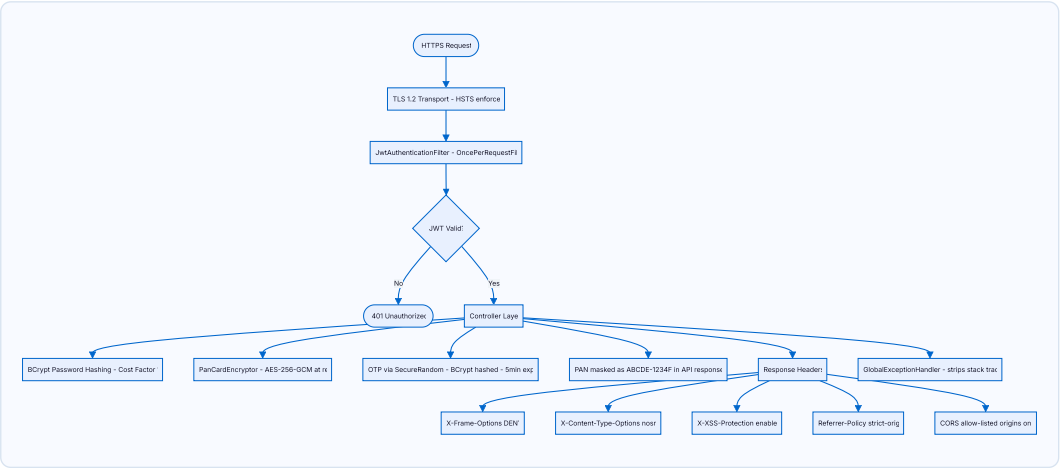
3. Caching Architecture



Design Rationale:

- NAV data is near-immutable once published (historical NAVs never change) → 7-day cache is safe
- Latest NAV updates once daily (after 11 PM IST when AMFI publishes) → 24-hour cache is optimal
- Caffeine eliminates 95%+ of external API calls, keeping mfapi.in within rate limits
- Write-through to PostgreSQL ensures data survives process restarts

4. Security Architecture



Key Security Decisions:

DECISION	RATIONALE
BCrypt over SHA-256 for passwords	BCrypt is adaptive (cost factor can increase over time); resistant to rainbow tables and GPU attacks

DECISION	RATIONALE
AES-256-GCM over AES-CBC for PAN	GCM provides authenticated encryption — detects tampering; no padding oracle vulnerabilities
Key derivation from JWT secret	Avoids introducing a second secret; SHA-256 hash of JWT secret produces a strong 256-bit AES key
Random IV per encryption	Prevents identical PAN values from producing identical ciphertexts
OTP hashed (not plaintext)	If DB is compromised, attacker cannot use stored OTPs to reset passwords
5-minute OTP expiry	Limits the attack window for intercepted OTPs

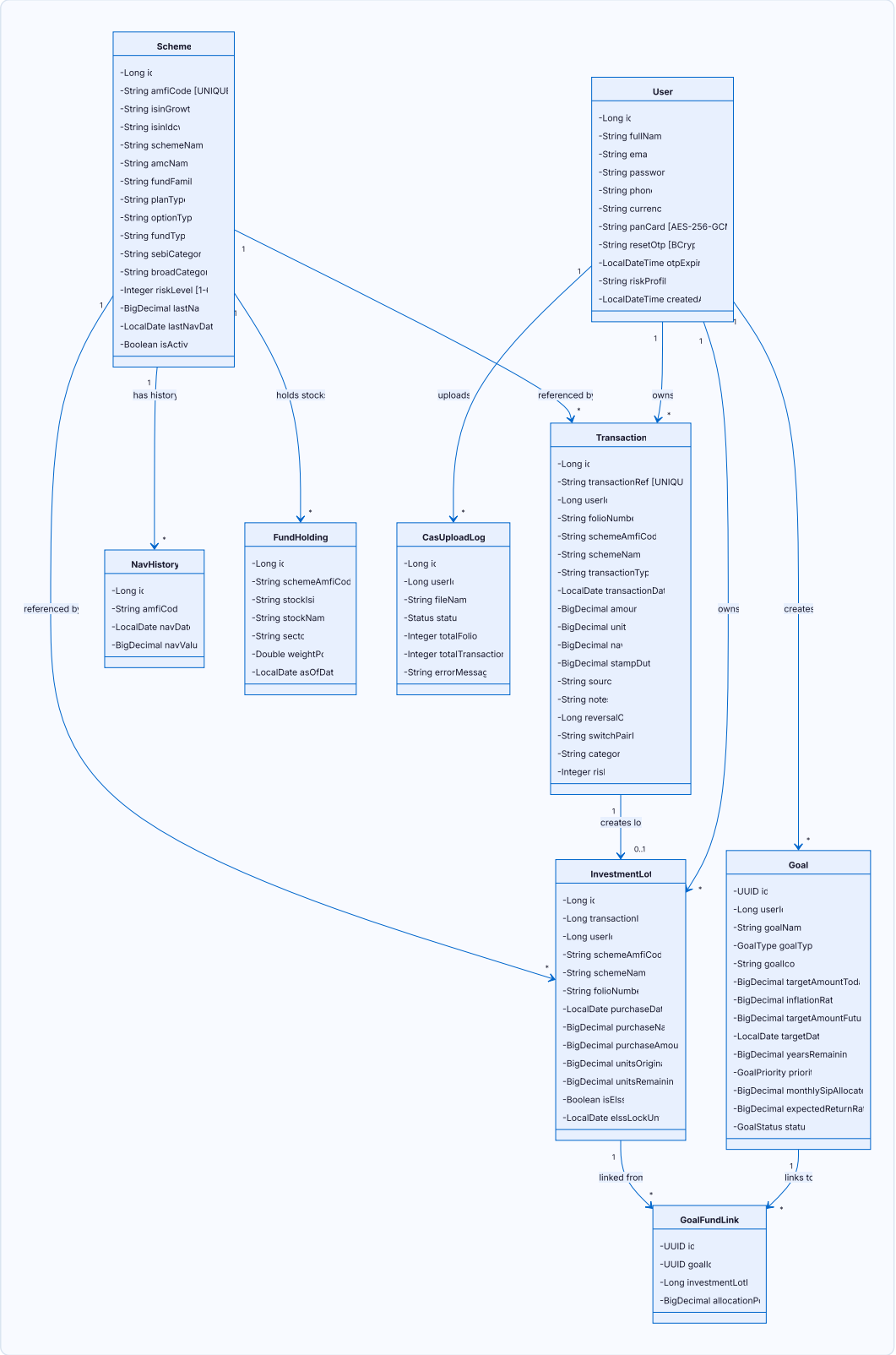
PART II: LOW-LEVEL DESIGN (LLD)

5. Module Breakdown

The backend is organized into 9 packages following the standard Spring Boot layered architecture:

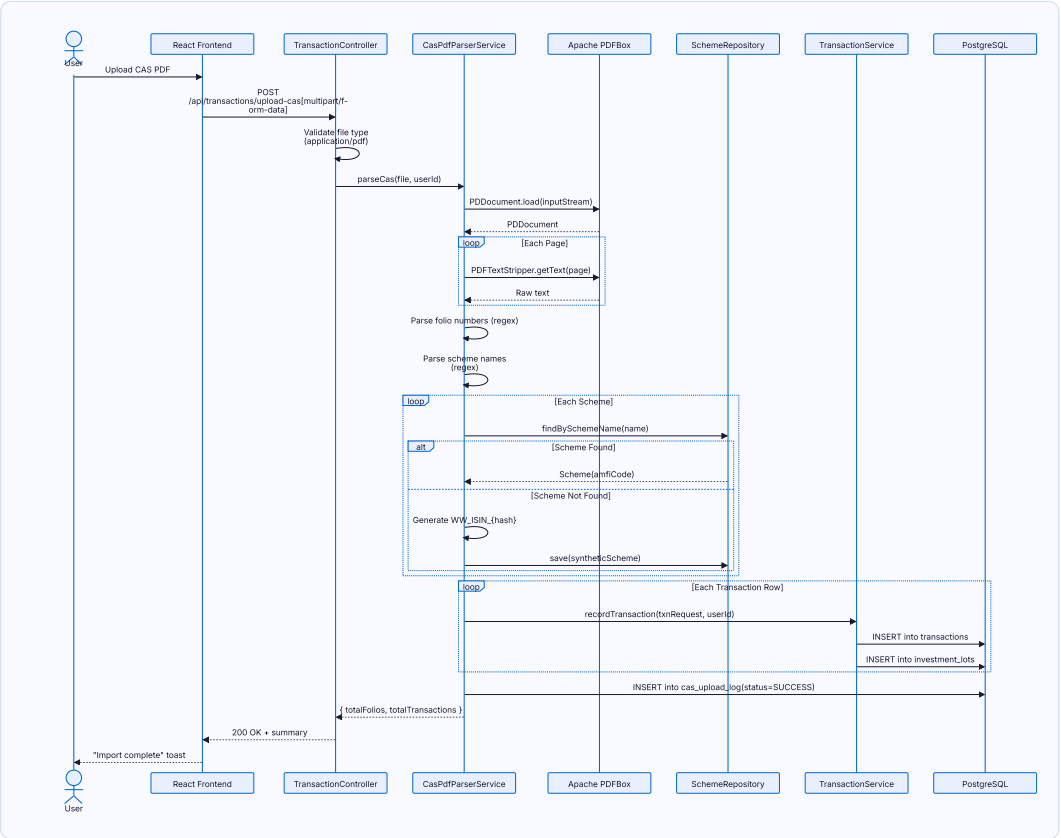
PACKAGE	CONTENTS	RESPONSIBILITY
`com.wealthwise`	`WealthWiseApplication.java`	Spring Boot entry point with `@SpringBootApplication`
`com.wealthwise.config`	`WebConfig.java`	CORS configuration, security response headers
`com.wealthwise.security`	`SecurityConfig`, `JwtService`, `JwtAuthenticationFilter`, `PanCardEncryptor`, `CacheConfig`	Authentication, authorization, encryption, caching
`com.wealthwise.controller`	10 controllers	REST API endpoint definitions
`com.wealthwise.service`	13 services	Business logic layer
`com.wealthwise.repository`	7 repositories	Spring Data JPA data access
`com.wealthwise.model`	7 entity classes	JPA entity definitions
`com.wealthwise.dto`	3 DTOs	Response transfer objects
`com.wealthwise.parser`	`NavAllTxtParser.java`	AMFI NAVAll.txt parsing logic
`com.wealthwise.util`	`TransactionTypeUtil.java`	Transaction type classification utilities

6. Class Diagram

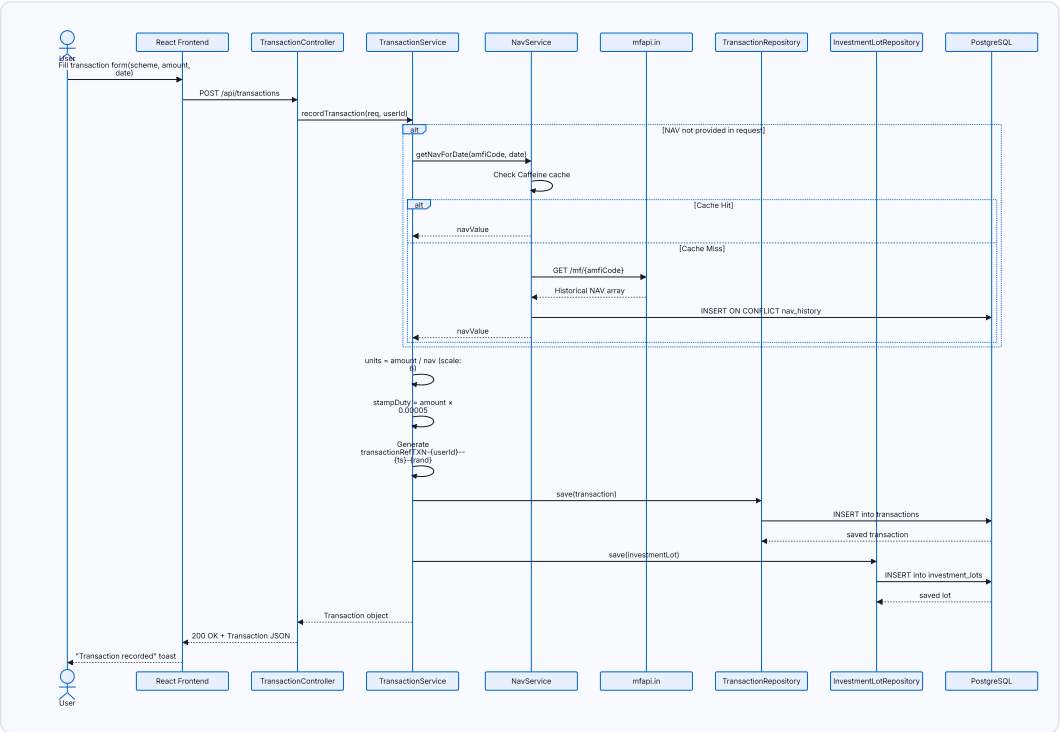


7. Sequence Diagrams

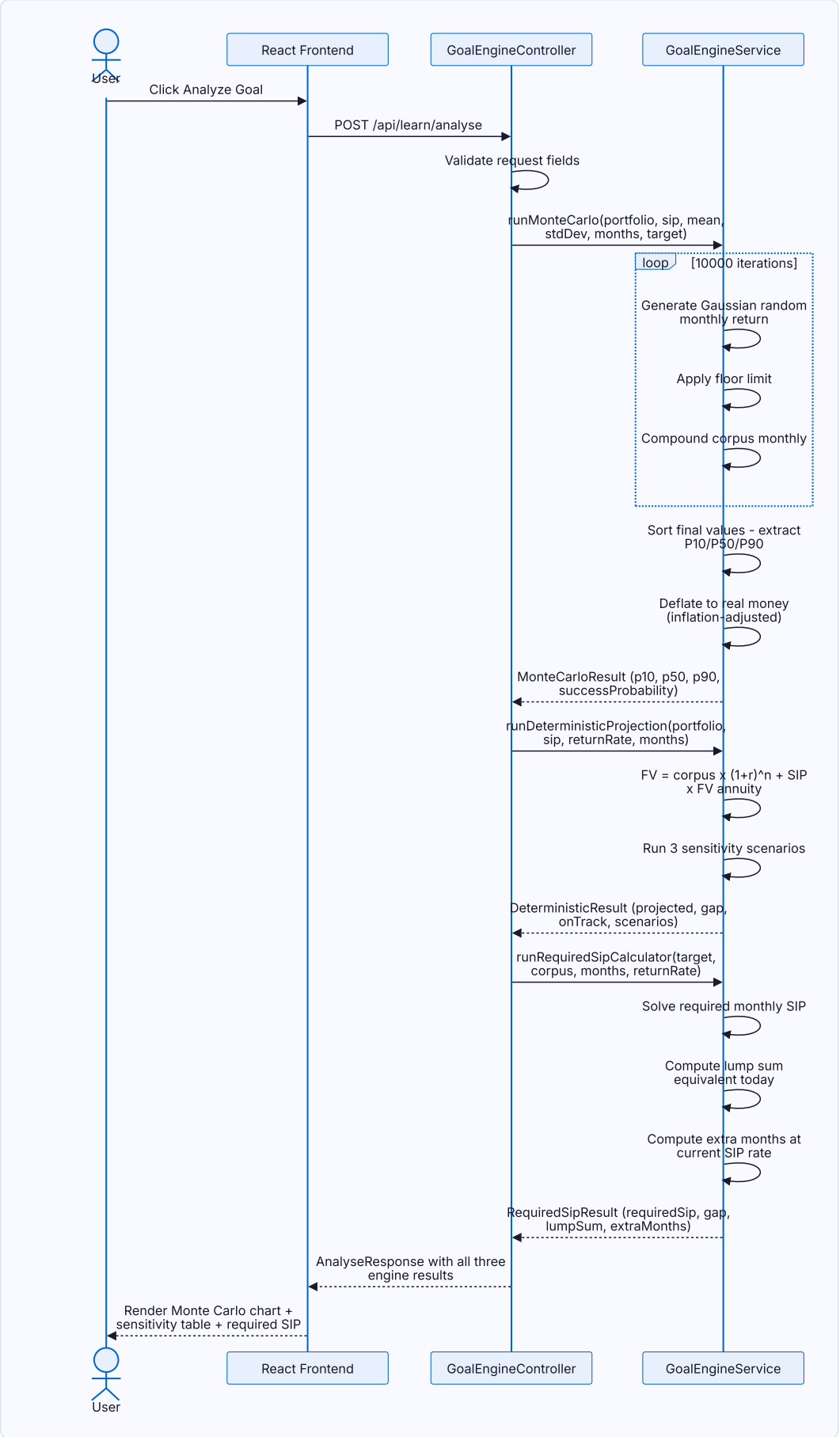
7.1 CAS PDF Import Flow



7.2 Transaction Recording Flow (Purchase with Lot Creation)

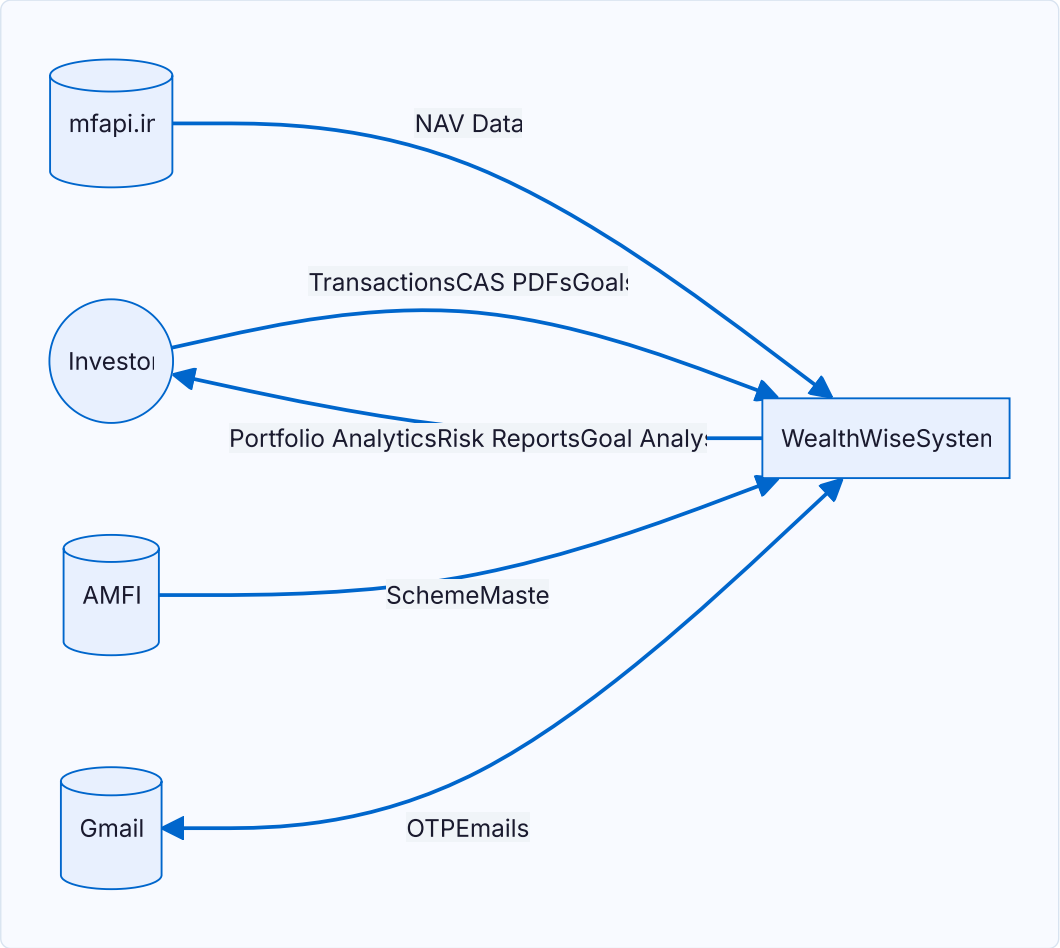


7.3 Goal Analysis Flow (Monte Carlo + Deterministic + Required SIP)

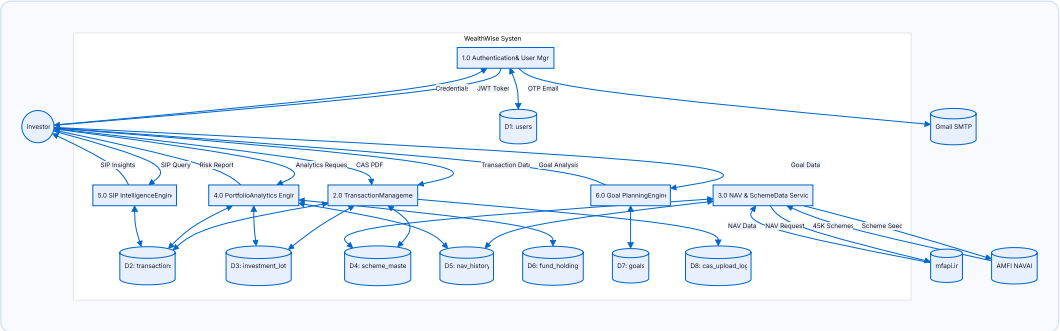


8. Data Flow Diagrams

8.1 DFD Level 0 (Context Diagram)

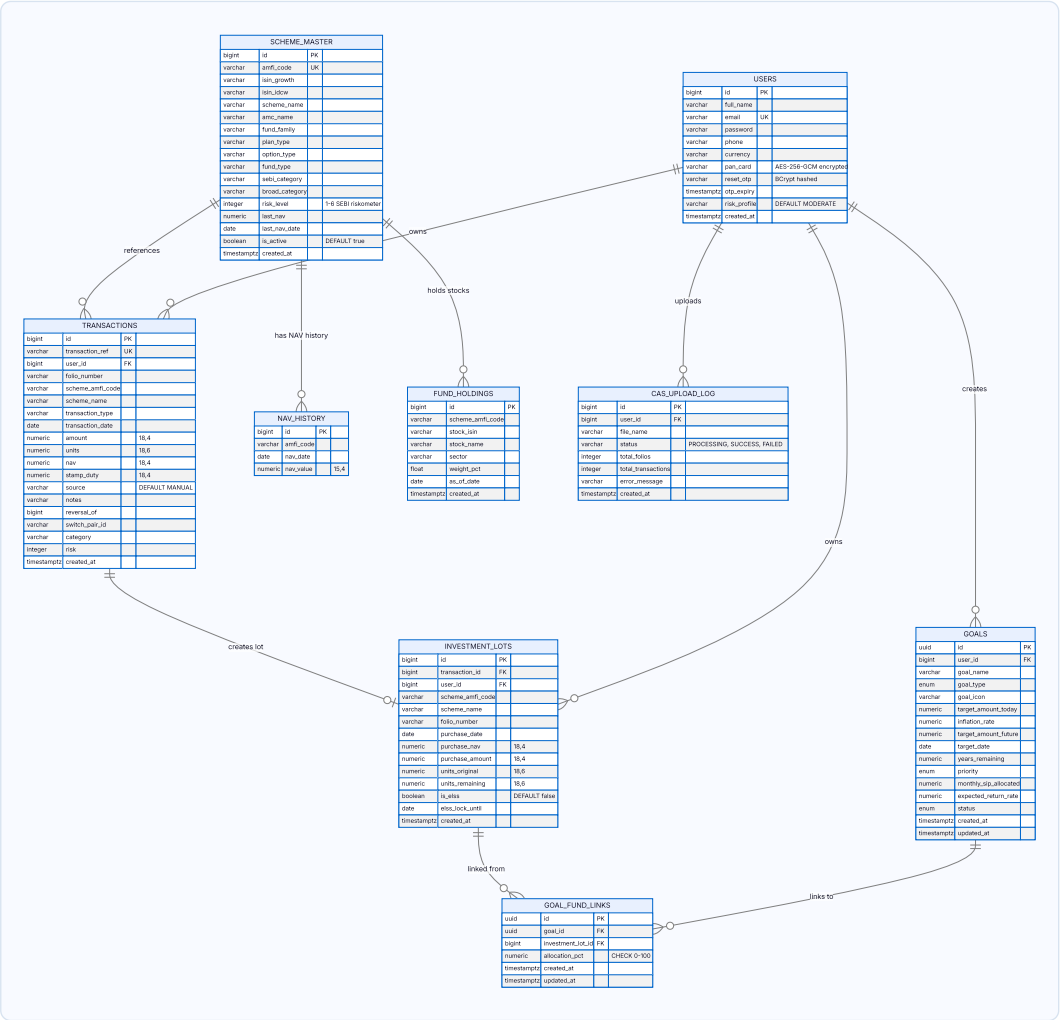


8.2 DFD Level 1



9. Database Schema Design

9.1 Entity-Relationship Diagram



9.2 Table Details and Constraints

TABLE	ROWS (EST.)	PRIMARY KEY	UNIQUE CONSTRAINTS	FOREIGN KEYS	KEY INDEXES
`users`	~10K	`id` (BIGSERIAL)	`email`	—	email
`transactions`	~500K	`id` (BIGSERIAL)	`transaction_ref`	`user_id` → users	(user_id), (user_id, scheme_amfi_code), (user_id, folio_number), (transaction_date)
`investment_lots`	~500K	`id` (BIGSERIAL)	—	`transaction_id` → transactions	(user_id), (user_id, scheme_amfi_code), (user_id, folio_number), (purchase_date)
`scheme_master`	~50K	`id` (BIGSERIAL)	`amfi_code`	—	(amfi_code), (scheme_name), (amc_name), (broad_category)

TABLE	ROWS (EST.)	PRIMARY KEY	UNIQUE CONSTRAINTS	FOREIGN KEYS	KEY INDEXES
`nav_history`	~10M	`id` (BIGSERIAL)	`(amfi_code, nav_date)`	—	(amfi_code, nav_date)
`fund_holdings`	~100K	`id` (BIGSERIAL)	—	—	(scheme_amfi_code), (stock_name)
`goals`	~50K	`id` (UUID)	—	`user_id` → users	(user_id)
`goal_fund_links`	~100K	`id` (UUID)	—	`goal_id` → goals, `investment_lot_id` → investment_lots	(goal_id), (investment_lot_id)
`cas_upload_log`	~20K	`id` (BIGSERIAL)	—	`user_id` → users	(user_id)

9.3 Precision Strategy

DATA TYPE	PRECISION	USAGE
`NUMERIC(18,4)`	18 total digits, 4 decimal places	Monetary amounts (INR), NAV values, stamp duty
`NUMERIC(18,6)`	18 total digits, 6 decimal places	Mutual fund units (MFs typically report to 3–4 decimals; 6 provides headroom)
`NUMERIC(15,4)`	15 total digits, 4 decimal places	Historical NAV values (max NAV ever ≈ ₹9,999,999.9999)

---

10. Detailed Function Logic / Pseudocode

10.1 XIRR Calculation (Newton-Raphson Method)

```
FUNCTION computeXIRR(cashflows: List<(date, amount)>) → Double:
    // cashflows: positive for inflows (purchases), negative for outflows (current value)

    // Initial guess
    rate ← 0.1  // 10% annual return

    FOR iteration = 1 TO 100:
        // Compute NPV at current rate
        npv ← 0
        npv_derivative ← 0
        base_date ← cashflows[0].date

        FOR EACH (date, amount) IN cashflows:
            years ← daysBetween(base_date, date) / 365.0
            npv ← npv + amount / (1 + rate)^years
            npv_derivative ← npv_derivative - years × amount / (1 + rate)^(years + 1)
        END FOR

        // Newton-Raphson update
        IF |npv_derivative| < 1e-10:
            BREAK  // derivative too small, stop

        new_rate ← rate - npv / npv_derivative

        // Convergence check
        IF |new_rate - rate| < 1e-7:
            RETURN new_rate

        rate ← new_rate
    END FOR

    RETURN rate  // best estimate after 100 iterations
```

10.2 Monte Carlo Simulation

```
FUNCTION runMonteCarlo(corpus, sip, μ, σ, months, target, inflation) → Result:
    monthlyInflation ← inflation / 12
    futureTarget ← target × (1 + monthlyInflation)^months

    finalValues ← empty list

    FOR sim = 1 TO 10,000:
        portfolio ← corpus

        FOR month = 1 TO months:
            // Generate random monthly return from Normal(μ, σ)
            r ← μ + σ × GaussianRandom()

            // Floor to prevent extreme negative compounding
            IF σ > 0.07: floor ← -0.30
            ELSE IF σ > 0.03: floor ← -0.20
            ELSE: floor ← -0.10

            r ← MAX(r, floor)
            portfolio ← portfolio × (1 + r) + sip
            portfolio ← MAX(portfolio, 0)  // cannot go negative
        END FOR

        finalValues.add(portfolio)
    END FOR

    SORT finalValues ASC

    // Extract percentiles and deflate to today's money
    p10 ← deflate(percentile(finalValues, 10), inflation, months)
    p50 ← deflate(percentile(finalValues, 50), inflation, months)
    p90 ← deflate(percentile(finalValues, 90), inflation, months)

    // Probability: count simulations exceeding FUTURE target (not deflated)
    probability ← COUNT(v ≥ futureTarget for v in finalValues) / 10,000 × 100

    RETURN { p10, p50, p90, probability }
```

10.3 FIFO Redemption Logic

```
FUNCTION processRedemption(userId, schemeCode, unitsToRedeem) → void:
    // Fetch active lots ordered by purchase date (oldest first = FIFO)
    lots ← InvestmentLotRepository.findByUserIdAndSchemeAmfiCode(
                userId, schemeCode, ORDER_BY purchase_date ASC)

    remaining ← unitsToRedeem

    FOR EACH lot IN lots:
        IF remaining ≤ 0: BREAK

        // Check ELSS lock-in
        IF lot.isElss AND lot.elssLockUntil > TODAY:
            CONTINUE  // skip locked lots

        IF lot.unitsRemaining ≥ remaining:
            // Partial redemption from this lot
            lot.unitsRemaining ← lot.unitsRemaining - remaining
            remaining ← 0
        ELSE:
            // Fully redeem this lot
            remaining ← remaining - lot.unitsRemaining
            lot.unitsRemaining ← 0
        END IF

        LotRepository.save(lot)
    END FOR

    IF remaining > 0:
        THROW IllegalStateException("Insufficient units: " + remaining + " units short")
```

10.4 CAS PDF Parsing Pipeline

```
FUNCTION parseCas(pdfFile, userId) → Result:
    document ← PDDocument.load(pdfFile.getInputStream())
    fullText ← PDFTextStripper.getText(document)

    // Phase 1: Extract folio blocks
    folioBlocks ← REGEX_SPLIT(fullText, "Folio No:\\s*(\\d+/\\d+)")

    result ← { totalFolios: 0, totalTransactions: 0, syntheticCodes: [] }

    FOR EACH (folioNumber, blockText) IN folioBlocks:
        result.totalFolios++

        // Phase 2: Extract scheme name from block header
        schemeName ← REGEX_MATCH(blockText, "^(.+?)\\s*-\\s*")

        // Phase 3: Resolve to AMFI code
        scheme ← SchemeRepository.findBySchemeName(schemeName)
        IF scheme IS NULL:
            // Try fuzzy match
            scheme ← SchemeRepository.findByNameContaining(normalize(schemeName))
        IF scheme IS NULL:
            // Generate synthetic code
            syntheticCode ← "WW_ISIN_" + MD5(schemeName).substring(0, 8)
            CREATE Scheme(amfiCode=syntheticCode, schemeName=schemeName)
            result.syntheticCodes.add(syntheticCode)

        // Phase 4: Parse transaction rows
        rows ← REGEX_FIND_ALL(blockText,
            "(\\d{2}-\\w{3}-\\d{4})\\s+(.+?)\\s+([\\d.]+)\\s+([\\d.]+)\\s+([\\d.]+)")

        FOR EACH (date, description, amount, nav, units) IN rows:
            txnType ← classifyTransactionType(description)
            TransactionService.recordTransaction({
                schemeAmfiCode, folioNumber, amount, nav, units,
                transactionDate: parseDate(date),
                transactionType: txnType,
                source: "CAS_IMPORT"
            }, userId)
            result.totalTransactions++
        END FOR
    END FOR

    // Phase 5: Audit log
    CasUploadLogRepository.save({
        userId, fileName: pdfFile.getOriginalFilename(),
        status: SUCCESS, totalFolios, totalTransactions
    })

    RETURN result
```

10.5 Fund Overlap Calculation

```
FUNCTION computeFundOverlapMatrix(userId) → OverlapMatrix:
    // Step 1: Get all distinct schemes in user's portfolio
    schemeCodes ← InvestmentLotRepository
        .findDistinctSchemeCodesByUserId(userId)
        .filter(code → NOT code.startsWith("WW_"))

    // Step 2: For each scheme, get stock holdings
    holdingsMap ← {}  // Map<schemeCode, Set<stockName>>
    FOR EACH code IN schemeCodes:
        holdings ← FundHoldingRepository.findBySchemeAmfiCode(code)
        IF holdings.isEmpty():
            // Trigger ingestion based on SEBI category
            FundHoldingsIngestionService.ingestForScheme(code)
            holdings ← FundHoldingRepository.findBySchemeAmfiCode(code)
        holdingsMap[code] ← holdings.map(h → h.stockName).toSet()
    END FOR

    // Step 3: Compute pairwise Jaccard overlap
    matrix ← []
    highOverlapPairs ← []

    FOR i = 0 TO schemeCodes.size() - 1:
        FOR j = i + 1 TO schemeCodes.size() - 1:
            setA ← holdingsMap[schemeCodes[i]]
            setB ← holdingsMap[schemeCodes[j]]

            intersection ← setA ∩ setB
            union ← setA ∪ setB

            overlapPct ← (intersection.size() / union.size()) × 100

            matrix.add({
                schemeA: schemeCodes[i],
                schemeB: schemeCodes[j],
                overlapPct: overlapPct,
                commonStocks: intersection.toList()
            })

            IF overlapPct > 30:
                highOverlapPairs.add({ pair, overlapPct, suggestion })
        END FOR
    END FOR

    RETURN { matrix, highOverlapPairs, consolidationSuggestions }
```

WealthWise — API Documentation

Base URL (Local): `http://localhost:8080`
Base URL (Production): `https://wealthwise-backend-zv5r.onrender.com`
API Version: v1 (implicit, no versioning prefix)
Content-Type: `application/json` (unless otherwise specified)

---

1. API Overview

The WealthWise API is a RESTful JSON API built on Spring Boot 3.2. All endpoints follow a consistent pattern:

- **Success:** HTTP 200 with JSON body
- **Client Error:** HTTP 400/401/404/413 with `{ "error": "message" }`
- **Server Error:** HTTP 500 with `{ "error": "An unexpected error occurred. Please try again later." }`

All responses include the following security headers:

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
Referrer-Policy: strict-origin-when-cross-origin
```

---

2. Authentication Mechanism

2.1 Token-Based Authentication (JWT)

WealthWise uses **JSON Web Tokens (JWT)** with HMAC-SHA256 signing for stateless authentication.

Token Lifecycle:

1. User signs up or signs in → Server generates JWT with `email` as subject
2. Client stores JWT in `localStorage` (key: `ww_token`)
3. Client includes JWT in all subsequent requests via the `Authorization` header
4. Server validates JWT signature and expiry on every protected request
5. On expiry, client detects it (either via API 401 response or client-side expiry check) and redirects to login

Header Format:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Token Payload (Decoded):

```
{
  "sub": "user@example.com",
  "iat": 1713091200,
  "exp": 1713177600
}
```

2.2 Public vs. Protected Endpoints

CATEGORY	PATH PREFIX	AUTHENTICATION
Public	`/api/auth/**`	None required
Public (Read-Only)	`GET /api/schemes/**`, `GET /api/nav/**`	None required

CATEGORY	PATH PREFIX	AUTHENTICATION
Protected	All other `/api/**` endpoints	JWT required

---

3. Endpoint Reference

3.1 Authentication (`/api/auth`)

POST `/api/auth/signup`

Auth: None
Description: Register a new user account.

Request Body:

```
{
  "fullName": "Bhuvan Nagesh",
  "email": "bhuvan@example.com",
  "password": "SecurePass123",
  "phone": "+919876543210",
  "currency": "INR",
  "panCard": "ABCDE1234F"
}
```

Response (200 OK):

```
{
  "token":
"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJiaHV2YW5AZXhhbXBsZS5jb20iLCJpYXQiOjE3MTMwOTEyMDAsImV4cCI6MTcxMzE3NzYwMH0.abc12
3",
  "user": {
    "id": 1,
    "fullName": "Bhuvan Nagesh",
    "email": "bhuvan@example.com",
    "password": null,
    "phone": "+919876543210",
    "currency": "INR",
    "panCard": "ABCDE****F",
    "riskProfile": "MODERATE",
    "createdAt": "2026-04-14T06:25:00"
  }
}
```

Error (400):

```
{ "error": "Email is already registered!" }
```

POST `/api/auth/signin`

Auth: None
Description: Authenticate with email and password.

Request Body:

```
{
  "email": "bhuvan@example.com",
  "password": "SecurePass123"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "user": { "id": 1, "fullName": "Bhuvan Nagesh", "email": "bhuvan@example.com", ... }
}
```

Error (401):

```
{ "error": "Invalid email or password" }
```

POST `/api/auth/forgot-password`

Auth: None
Description: Generate and send a 6-digit OTP to the user's email for password reset.

Request Body:

```
{ "email": "bhuvan@example.com" }
```

Response (200 OK):

```
{ "message": "OTP generated and sent to email successfully." }
```

Error (400):

```
{ "error": "No account found with this email" }
```

POST `/api/auth/reset-password`

Auth: None
Description: Verify OTP and set a new password.

Request Body:

```
{
  "email": "bhuvan@example.com",
  "otp": "482931",
  "newPassword": "NewSecurePass456"
}
```

Response (200 OK):

```
{ "message": "Password has been successfully reset." }
```

Errors:

```
{ "error": "Invalid or missing OTP" }
{ "error": "OTP has expired. Please request a new one." }
```

GET `/api/auth/health`

Auth: None
Description: Health check endpoint used by Render for health monitoring and frontend warmup.

Response (200 OK):

```
{ "status": "UP" }
```

3.2 User Profile (`/api/user`)

GET `/api/user/profile`

Auth: JWT (via `Authorization` header)
Description: Get the authenticated user's profile. PAN card is masked in response.

Response (200 OK):

```
{
  "id": 1,
  "fullName": "Bhuvan Nagesh",
  "email": "bhuvan@example.com",
  "password": null,
  "phone": "+919876543210",
  "currency": "INR",
  "panCard": "ABCDE****F",
  "riskProfile": "MODERATE",
  "createdAt": "2026-04-14T06:25:00"
}
```

PUT `/api/user/profile`

Auth: JWT
Description: Update user profile fields.

Request Body:

```
{
  "fullName": "Bhuvan N",
  "phone": "+919876543211",
  "currency": "USD",
  "panCard": "FGHIJ5678K"
}
```

Response (200 OK):

```
{
  "message": "Profile updated successfully",
  "user": { "id": 1, "fullName": "Bhuvan N", "panCard": "FGHIJ****K", ... }
}
```

POST `/api/user/change-password`

Auth: JWT
Description: Change password with current password verification.

Request Body:

```
{
  "currentPassword": "OldPass123",
  "newPassword": "NewPass456"
}
```

Response (200 OK):

```
{ "message": "Password changed successfully" }
```

Errors:

```
{ "error": "currentPassword and newPassword are required" }
{ "error": "New password must be at least 8 characters" }
{ "error": "Current password is incorrect" }
```

3.3 Scheme Master (`/api/schemes`)

GET `/api/schemes/search`

Auth: None (public)
Description: Search schemes with optional filters.

Query Parameters:

PARAM	TYPE	DEFAULT	DESCRIPTION
`q`	String	`""`	Search term (scheme name, AMC name)
`category`	String	null	Filter by broad category (EQUITY, DEBT, HYBRID)
`planType`	String	null	Filter by plan type (DIRECT, REGULAR)
`page`	int	0	Page number (0-indexed)
`size`	int	10	Page size

Example: `GET /api/schemes/search?q=axis+bluechip&category=EQUITY&planType=DIRECT&page=0&size=10`

Response (200 OK):

```
{
  "content": [
    {
      "id": 12345,
      "amfiCode": "120503",
      "isinGrowth": "INF846K01EW2",
      "schemeName": "Axis Bluechip Fund - Direct Plan - Growth Option",
      "amcName": "Axis Mutual Fund",
      "planType": "DIRECT",
      "optionType": "GROWTH",
      "fundType": "OPEN_ENDED",
      "sebiCategory": "Large Cap Fund",
      "broadCategory": "EQUITY",
      "riskLevel": 5,
      "lastNav": 62.34,
      "lastNavDate": "2026-04-11",
      "isActive": true
    }
  ],
  "totalElements": 1,
  "totalPages": 1,
  "page": 0
}
```

GET `/api/schemes/{amfiCode}`

Auth: None (public)
Description: Get scheme details by AMFI code.

Response (200 OK): Full scheme object (same structure as search content item)
Response (404): Empty body

GET `/api/schemes/nav/{amfiCode}`

Auth: None (public)
Description: Get the latest NAV for a scheme from the scheme_master table.

Response (200 OK):

```
{
  "amfiCode": "120503",
  "schemeName": "Axis Bluechip Fund - Direct Plan - Growth Option",
  "nav": 62.34,
  "navDate": "2026-04-11"
}
```

GET `/api/schemes/count`

Auth: None (public)
Description: Get total count of active schemes in the database.

Response (200 OK):

```
{ "activeSchemes": 45823 }
```

POST `/api/schemes/seed`

Auth: None (public, admin use)
Description: Trigger scheme master seeding from AMFI NAVAll.txt.

Response (200 OK):

```
{ "totalProcessed": 45823, "newInserted": 127, "updated": 45696 }
```

3.4 NAV Data (`/api/nav`)

GET `/api/nav/latest/{amfiCode}`

Auth: None (public)
Description: Get the latest NAV for a scheme. Cached for 24 hours in Caffeine.

Response (200 OK):

```
{
  "amfiCode": "120503",
  "schemeName": "Axis Bluechip Fund - Direct Plan - Growth Option",
  "nav": "62.3400",
  "date": "11-04-2026"
}
```

POST `/api/nav/refresh/{amfiCode}`

Auth: None (public)
Description: Force-refresh NAV from mfapi.in, bypassing cache.

Response (200 OK): Same format as latest NAV

GET `/api/nav/{amfiCode}/date/{dateStr}`

Auth: None (public)
Description: Get NAV for a specific date. Date format: `yyyy-MM-dd` or `dd-MM-yyyy`.

Response (200 OK — NAV found):

```
{ "amfiCode": "120503", "date": "2026-04-11", "nav": 62.34 }
```

Response (200 OK — NAV not available, holiday):

```
{
  "amfiCode": "120503",
  "date": "2026-04-13",
  "nav": null,
  "message": "NAV not available for this date (possible holiday). Use nearest available NAV."
}
```

GET `/api/nav/{amfiCode}/history`

Auth: None (public)

Description: Get full historical NAV data for a scheme. Cached for 7 days.

Response (200 OK):

```
[
  { "date": "11-04-2026", "nav": "62.3400" },
  { "date": "10-04-2026", "nav": "61.9800" },
  ...
]
```

GET `/api/nav/synthetic-schemes`

Auth: None (public)

Description: List all synthetic WW_ISIN_* scheme codes currently in the database.

Response (200 OK):

```
{
  "count": 3,
  "schemes": [
    { "syntheticCode": "WW_ISIN_a1b2c3d4", "schemeName": "Some Scheme Name", "isin": "" }
  ]
}
```

POST `/api/nav/reconcile-synthetic-schemes`

Auth: None (public, admin use)

Description: Reconcile all synthetic codes to real AMFI codes. Idempotent.

Response (200 OK):

```
{
  "totalSynthetic": 3,
  "resolved": 2,
  "failed": 1,
  "details": [
    { "from": "WW_ISIN_a1b2c3d4", "to": "120503", "status": "RESOLVED" },
    { "from": "WW_ISIN_e5f6g7h8", "to": null, "status": "UNRESOLVED" }
  ]
}
```

3.5 Transactions (`/api/transactions`)

POST `/api/transactions`

Auth: JWT

Description: Record a new mutual fund transaction.

Request Body:

```
{
  "schemeAmfiCode": "120503",
  "schemeName": "Axis Bluechip Fund - Direct Plan - Growth Option",
  "folioNumber": "12345678/01",
  "transactionType": "PURCHASE_SIP",
  "transactionDate": "2026-04-01",
  "amount": 5000.00,
  "nav": null,
  "units": null,
  "notes": "April SIP"
}
```

> ***Note:*** *If `nav` is null, the system auto-fetches NAV for the given date from mfapi.in. If `units` is null, units are computed as `amount / nav`.*

Response (200 OK):

```
{
  "id": 42,
  "transactionRef": "TXN-1-1713091200000-a7b3",
  "userId": 1,
  "schemeAmfiCode": "120503",
  "schemeName": "Axis Bluechip Fund - Direct Plan - Growth Option",
  "folioNumber": "12345678/01",
  "transactionType": "PURCHASE_SIP",
  "transactionDate": "2026-04-01",
  "amount": 5000.0000,
  "units": 80.231200,
  "nav": 62.3200,
  "stampDuty": 0.2500,
  "source": "MANUAL",
  "notes": "April SIP",
  "createdAt": "2026-04-14T06:25:00"
}
```

Supported Transaction Types:

TYPE	DESCRIPTION	LOT EFFECT
`PURCHASE_LUMPSUM`	One-time purchase	Creates new lot
`PURCHASE_SIP`	SIP installment	Creates new lot
`REDEMPTION`	Sell units	Reduces lot units (FIFO)
`SWITCH_IN`	Transfer into fund	Creates new lot
`SWITCH_OUT`	Transfer out of fund	Reduces lot units
`SWP`	Systematic withdrawal	Reduces lot units
`STP_IN`	Systematic transfer in	Creates new lot
`STP_OUT`	Systematic transfer out	Reduces lot units
`DIVIDEND_PAYOUT`	Cash dividend	No lot effect
`DIVIDEND_REINVEST`	Dividend reinvested	Creates new lot
`REVERSAL`	Undo a previous transaction	Reverses original lot effect

---

POST `/api/transactions/bulk-sip`

Auth: JWT

Description: Generate multiple monthly SIP transactions at once.

Request Body:

```
{
  "schemeAmfiCode": "120503",
  "schemeName": "Axis Bluechip Fund - Direct Plan - Growth Option",
  "folioNumber": "12345678/01",
  "amount": 5000.00,
  "startDate": "2025-01-01",
  "endDate": "2026-03-01"
}
```

Response (200 OK):

```
{
  "message": "Successfully generated 15 SIP transactions",
  "transactions": [ /* array of 15 Transaction objects */ ]
}
```

Error (400):

```
{ "error": "Bulk SIP range cannot exceed 120 months (10 years). Requested: 150 months." }
```

POST `/api/transactions/upload-cas`

Auth: JWT
Content-Type: `multipart/form-data`
Description: Upload and parse a CAS PDF statement.

Request: Form data with `file` field containing a PDF file.

Response (200 OK):

```
{
  "totalFolios": 3,
  "totalTransactions": 47,
  "syntheticCodes": ["WW_ISIN_a1b2c3d4"],
  "status": "SUCCESS"
}
```

GET `/api/transactions`

Auth: JWT
Description: Get all transactions for the authenticated user.

Response (200 OK): Array of Transaction objects, ordered by date descending.

GET `/api/transactions/{id}`

Auth: JWT
Description: Get a single transaction by ID (must belong to the authenticated user).

Response (200 OK): Transaction object
Response (404): Empty body

POST `/api/transactions/{id}/reverse`

Auth: JWT
Description: Create a reversal transaction for the specified transaction.

Response (200 OK): The newly created reversal Transaction object

---

GET `/api/transactions/portfolio-summary`

Auth: JWT
Description: Get per-scheme portfolio summary with current valuations.

Response (200 OK):

```
[
  {
    "schemeAmfiCode": "120503",
    "schemeName": "Axis Bluechip Fund - Direct Plan - Growth Option",
    "totalUnits": 803.45,
    "totalInvested": 50000.00,
    "currentNav": 62.34,
    "currentValue": 50089.50,
    "pnl": 89.50,
    "returnPct": 0.18
  }
]
```

GET `/api/transactions/by-scheme/{amfiCode}`

Auth: JWT
Description: Get all transactions for a specific scheme.

3.6 Analytics (`/api/analytics`)

GET `/api/analytics/risk`

Auth: JWT
Description: Get portfolio risk profile including volatility, Sharpe ratio, and SEBI risk score.

Response (200 OK):

```
{
  "portfolioVolatility": 0.1842,
  "sharpeRatio": 0.72,
  "weightedRiskScore": 4.3,
  "riskCategory": "MODERATE",
  "perSchemeRisk": [
    {
      "schemeAmfiCode": "120503",
      "schemeName": "Axis Bluechip Fund",
      "volatility": 0.1623,
      "sharpeRatio": 0.81,
      "riskLevel": 5,
      "allocationPct": 60.0
    }
  ]
}
```

GET `/api/analytics/sip`

Auth: JWT
Description: Get SIP intelligence summary.

GET `/api/analytics/overlap`

Auth: JWT
Description: Get fund overlap matrix with stock-level intersection analysis.

Response (200 OK):

```
{
  "overlapMatrix": [
    {
      "schemeA": "120503",
      "schemeAName": "Axis Bluechip Fund",
      "schemeB": "119551",
      "schemeBName": "SBI Bluechip Fund",
      "overlapPct": 68.5,
      "commonStocks": ["Reliance Industries", "HDFC Bank", "Infosys", "TCS"]
    }
  ],
  "highOverlapPairs": 2,
  "consolidationSuggestions": [
    "Axis Bluechip and SBI Bluechip have 68.5% overlap — consider consolidating into one"
  ]
}
```

PATCH `/api/analytics/risk-profile`

Auth: JWT
Description: Manually update the user's risk profile.

Request Body:

```
{ "riskProfile": "AGGRESSIVE" }
```

Response (200 OK):

```
{ "message": "Risk profile updated to AGGRESSIVE" }
```

3.7 SIP Intelligence (`/api/sip`)

GET `/api/sip/dashboard`

Auth: JWT
Description: Get SIP overview dashboard.

Response (200 OK):

```
{
  "totalActiveSIPs": 4,
  "monthlyOutflow": 25000.0,
  "nextStepUpDate": "2027-01-01",
  "projectedAmount": 1245000.0,
  "sipStreak": "14 months",
  "alert": null
}
```

GET `/api/sip/compare`

Auth: JWT
Description: Compare aggregate SIP returns vs. hypothetical lumpsum.

Response (200 OK):

```
{
  "sipValue": 128500.0,
  "lumpsumValue": 134200.0,
  "sipXirr": 14.2,
  "lumpsumReturn": 18.6,
  "winner": "LUMPSUM",
  "reason": "In a rising market, lumpsum outperforms SIP by ₹5,700"
}
```

GET `/api/sip/optimize`

Auth: JWT
Description: Recommend the optimal SIP day based on historical NAV patterns.

Response (200 OK):

```
{
  "bestDay": 7,
  "avgNavOnBestDay": 58.23,
  "avgNavOnCurrentDay": 59.01,
  "potentialSaving": "1.3%",
  "recommendation": "Switching your SIP date from the 1st to the 7th could save ~1.3% on average"
}
```

GET `/api/sip/topup`

Auth: JWT
Description: SIP step-up projection with and without annual increases.

Response (200 OK):

```
{
  "currentSipMonthly": 25000,
  "withoutStepUp": {
    "years": 10,
    "totalInvested": 3000000,
    "projectedValue": 5640000
  },
  "withStepUp10Pct": {
    "years": 10,
    "totalInvested": 4780000,
    "projectedValue": 8920000,
    "additionalGain": 3280000
  }
}
```

3.8 Returns (`/api/returns`)

GET `/api/returns/portfolio`

Auth: JWT
Description: Get portfolio-level returns with growth timeline.

Response (200 OK):

```
{
  "totalInvested": 500000,
  "currentValue": 628000,
  "absoluteReturn": 128000,
  "absoluteReturnPct": 25.6,
  "xirr": 18.3,
  "growthTimeline": [
    { "date": "2025-01", "value": 50000 },
    { "date": "2025-02", "value": 102000 },
    ...
  ]
}
```

GET `/api/returns/scheme/{amfiCode}`

Auth: JWT
Description: Get returns for a specific scheme.

3.9 Goal Engine (`/api/learn`)

POST `/api/learn/analyse`

Auth: None (stateless computation endpoint)
Description: Run combined goal analysis: Monte Carlo, Deterministic Projection, and Required SIP Calculator. All output values are in today's money (inflation-adjusted).

Request Body:

```
{
  "initialPortfolio": 500000,
  "monthlyContribution": 25000,
  "monthlyMean": 0.01,
  "monthlyStdDev": 0.04,
  "months": 240,
  "targetAmount": 50000000,
  "annualInflationRate": 0.06
}
```

Validation Constraints:

FIELD	CONSTRAINT
`initialPortfolio`	`@Positive` (> 0)
`monthlyContribution`	`@PositiveOrZero` (≥ 0)
`monthlyStdDev`	`@Positive` (> 0)
`months`	`@Min(1) @Max(600)`
`targetAmount`	`@Positive` (> 0)
`annualInflationRate`	`@DecimalMin("0.0") @DecimalMax("1.0")`

Response (200 OK):

```
{
  "monteCarlo": {
    "pessimistic": 21000000,
    "likely": 48000000,
    "optimistic": 92000000,
    "probability": 42.5
  },
  "deterministic": {
    "fvCorpus": 8200000,
    "fvSip": 38000000,
    "totalProjected": 46200000,
    "gap": 3800000,
    "onTrack": false,
    "sensitivity": [
      { "scenario": "Return = 10% pa", "projected": 32000000, "gap": 18000000 },
      { "scenario": "6 SIPs missed", "projected": 43500000, "gap": 6500000 },
      { "scenario": "Inflation at 8%", "projected": 38000000, "gap": 12000000 }
    ]
  },
  "requiredSip": {
    "requiredSip": 32000,
    "currentSip": 25000,
    "sipGap": 7000,
    "lumpSumToday": 1800000,
    "extraMonths": 36,
    "currentSipEnough": false
  }
}
```

4. Global Error Codes

HTTP CODE	MEANING	WHEN RETURNED
200	Success	All successful operations
400	Bad Request	Validation failure, business rule violation, missing required fields
401	Unauthorized	Missing/invalid/expired JWT token
404	Not Found	Resource does not exist (scheme, transaction)
413	Payload Too Large	CAS PDF exceeds 10 MB upload limit
500	Internal Server Error	Unhandled server exception (message sanitized, no stack traces)

5. Rate Limiting & Caching Notes

CONCERN	STRATEGY
mfapi.in Rate Limiting	Caffeine cache at `nav_latest` (24h TTL) and `nav_history` (7d TTL) reduces external calls by 95%+
Scheme Search Performance	PostgreSQL B-tree indexes on `scheme_name`, `amfi_code`, `broad_category` ensure <500ms response
CAS Upload Protection	Max file size: 10 MB (enforced by Spring Boot `multipart.max-file-size`)
Bulk SIP Protection	Hard cap at 120 months (10 years) per request to prevent abuse
JWT Expiry	Configurable via `app.jwt.expiration-ms` environment variable

WealthWise — Deployment Guide

1. Prerequisites

1.1 Software Requirements

SOFTWARE	MINIMUM VERSION	RECOMMENDED VERSION	PURPOSE
Java JDK	17	Eclipse Temurin 17.0.x	Backend compilation and runtime
Apache Maven	3.8.x	3.8.7+	Backend dependency management and build
Node.js	18.x	22.x LTS	Frontend build tooling (Vite)
npm	9.x	10.x+	Frontend package management
PostgreSQL	14.x	15.x	Database (or use Supabase cloud)
Git	2.30+	Latest	Source code management
Docker	20.10+	24.x+	Container builds (production)

1.2 Accounts Required

SERVICE	PURPOSE	URL
Supabase	Managed PostgreSQL hosting	https://supabase.com
Render	Cloud deployment (backend + frontend)	https://render.com
Gmail	SMTP email for OTP delivery	Requires App Password
GitHub	Source code repository	https://github.com

1.3 Verify Installation

```
# Verify Java
java -version
# Expected: openjdk version "17.0.x"

# Verify Maven
mvn -version
# Expected: Apache Maven 3.8.x

# Verify Node.js
node -v
# Expected: v22.x.x

# Verify npm
npm -v
# Expected: 10.x.x

# Verify Docker (if building containers)
docker --version
# Expected: Docker version 24.x.x
```

2. Environment Configuration

2.1 Backend Environment Variables

The backend reads configuration from environment variables (not committed to version control). A template is provided at `backend/.env.local.example`.

VARIABLE	REQUIRED	DESCRIPTION	EXAMPLE VALUE
`DB_URL`	✅	JDBC connection string to PostgreSQL database. Must include SSL mode for Supabase.	`jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require`
`DB_USERNAME`	✅	Database username	`postgres`
`DB_PASSWORD`	✅	Database password	`your-secure-password`
`JWT_SECRET`	✅	HMAC-SHA256 signing key for JWT tokens. Must be ≥32 characters. Also used to derive AES-256 key for PAN encryption.	`WealthWise$ecretKey2026!SuperSecureRandomString#MF`
`MAIL_USERNAME`	✅	Gmail address for sending OTP emails	`yourname@gmail.com`
`MAIL_PASSWORD`	✅	Gmail App Password (not regular password). Generate at https://myaccount.google.com/apppasswords	`abcd efgh ijkl mnop`
`CORS_ALLOWED_ORIGINS`	⚠️	Comma-separated list of allowed frontend origins. Defaults to localhost if not set.	`http://localhost:5173,https://your-frontend.onrender.com`
`SPRING_PROFILES_ACTIVE`	⚠️	Spring profile. Use `prod` for production deployment on Render.	`prod`
`PORT`	⚠️	Server port. Render sets this automatically. Default: 8080	`8080`

2.2 Frontend Environment Variables

The frontend uses Vite's `import.meta.env` system. Variables are read from `.env` files in the `project/` directory.

VARIABLE	FILE	DESCRIPTION	EXAMPLE VALUE
`VITE_API_BASE_URL`	`.env` (dev), `.env.production` (prod)	Backend API base URL (no trailing slash)	`http://localhost:8080` (dev) / `https://wealthwise-backend-zv5r.onrender.com` (prod)

2.3 Security Notes

> ⚠️ ***CRITICAL:*** *Never commit secrets to version control. The following files are in* `.gitignore`*:*
>
> - `backend/src/main/resources/application.properties`
> - `backend/src/main/resources/application-local.properties`
> - `project/.env`
> - `project/.env.local`
> - `project/.env.*.local`

3. Local Development Setup

3.1 Clone the Repository

```
git clone https://github.com/your-username/Wealthwise.git
cd Wealthwise
```

3.2 Backend Setup

Step 1: Configure Environment Variables

```
# Copy the example environment file
cd backend
copy .env.local.example .env.local
```

Edit `backend/.env.local` with your actual values:

```
DB_URL=jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require
DB_USERNAME=postgres
DB_PASSWORD=your-password
JWT_SECRET=your-32-character-minimum-secret-key-here
MAIL_USERNAME=yourname@gmail.com
MAIL_PASSWORD=your-app-password
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000
```

Step 2: Create application.properties

Create `backend/src/main/resources/application.properties`:

```
# ── Database ──────────────────────────────────────
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

# ── JPA / Hibernate ───────────────────────────────
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.properties.hibernate.format_sql=true

# ── JWT ───────────────────────────────────────────
app.jwt.secret=${JWT_SECRET}
app.jwt.expiration-ms=86400000

# ── Mail (Gmail SMTP) ─────────────────────────────
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

# ── CORS ──────────────────────────────────────────
cors.allowed.origins=${CORS_ALLOWED_ORIGINS:http://localhost:5173}

# ── File Upload ───────────────────────────────────
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB

# ── Server ────────────────────────────────────────
server.port=${PORT:8080}
```

Step 3: Set Environment Variables (Windows PowerShell)

```
$env:DB_URL = "jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require"
$env:DB_USERNAME = "postgres"
$env:DB_PASSWORD = "your-password"
$env:JWT_SECRET = "WealthWise-SecretKey2026-SuperSecure"
$env:MAIL_USERNAME = "yourname@gmail.com"
$env:MAIL_PASSWORD = "your-app-password"
$env:CORS_ALLOWED_ORIGINS = "http://localhost:5173"
```

Step 4: Build and Run

```
cd backend

# Download dependencies (first time only)
mvn dependency:resolve

# Build the project (skip tests for speed)
mvn clean package -DskipTests

# Run the backend
mvn spring-boot:run

# Or run the JAR directly
java -jar target/wealthwise-backend-0.0.1-SNAPSHOT.jar
```

The backend will start on `http://localhost:8080`. Verify with:

```
curl http://localhost:8080/api/auth/health
# Expected: {"status":"UP"}
```

3.3 Frontend Setup

```
cd project

# Install dependencies
npm install

# Create .env with local backend URL
echo "VITE_API_BASE_URL=http://localhost:8080" > .env

# Start development server with HMR
npm run dev
```

The frontend will start on `http://localhost:5173`.

3.4 Database Setup

If using **Supabase** (recommended):

1. Create a project at https://supabase.com
2. Go to Settings → Database → Connection String → URI
3. Use the JDBC format: `jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require`
4. Spring Boot's `ddl-auto=update` will create tables automatically on first run

If using **local PostgreSQL**:

```
CREATE DATABASE wealthwise;
-- Tables are auto-created by JPA/Hibernate on first run
```

3.5 Seed Scheme Data

After the backend is running, seed the scheme master database:

```
# This fetches ~45,000 schemes from AMFI and seeds the scheme_master table
curl -X POST http://localhost:8080/api/schemes/seed
```

> ***Note:*** *This operation takes 30–60 seconds and should be run once after initial setup.*

---

4. Production Deployment (Render)

4.1 Repository Structure for Render

Render expects the following structure (matching the existing `render.yaml`):

```
Wealthwise/
├── backend/
│   ├── Dockerfile          ← Render builds the backend from this
│   ├── pom.xml
│   └── src/
├── project/
│   ├── package.json        ← Render runs `npm ci && npm run build`
│   └── src/
└── render.yaml             ← Infrastructure-as-code deployment spec
```

4.2 Deploy Backend (Docker Web Service)

1. **Connect Repository:** In Render dashboard → New → Web Service → Connect GitHub repo
2. **Configuration:**
   - **Name:** `wealthwise-backend`
   - **Runtime:** Docker
   - **Dockerfile Path:** `./backend/Dockerfile`
   - **Docker Context:** `./backend`
   - **Plan:** Free
   - **Health Check Path:** `/api/auth/health`
3. **Environment Variables:** Set in Render's Environment tab:

KEY	VALUE
`SPRING_PROFILES_ACTIVE`	`prod`
`PORT`	`8080`
`DB_URL`	`jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require`
`DB_USERNAME`	`postgres`
`DB_PASSWORD`	`<your supabase password>`
`JWT_SECRET`	`<min 32-char random string>`
`MAIL_USERNAME`	`yourname@gmail.com`
`MAIL_PASSWORD`	`<gmail app password>`
`CORS_ALLOWED_ORIGINS`	`https://your-frontend.onrender.com`

4. **Docker Build Process:**

```
# Stage 1: Build (Maven + JDK 17)
FROM maven:3.8.7-eclipse-temurin-17 AS build
COPY pom.xml .
RUN mvn dependency:go-offline -B  # Cached layer
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run (JRE 17 only — smaller image)
FROM eclipse-temurin:17-jre-jammy
COPY --from=build /app/target/*.jar app.jar
ENTRYPOINT ["java", "-XX:TieredStopAtLevel=1", "-XX:MaxRAMPercentage=70.0",
            "-XX:+UseSerialGC", "-Xss512k", "-jar", "app.jar"]
```

4.3 Deploy Frontend (Static Site)

1. **In Render dashboard:** New → Static Site → Connect GitHub repo
2. **Configuration:**
   - **Name:** `wealthwise-frontend`
   - **Build Command:** `cd project && npm ci && npm run build`
   - **Publish Directory:** `./project/dist`
3. **Rewrite Rule:** Add a rewrite `/* → /index.html` (SPA routing support)
4. **Environment Variables:**

KEY	VALUE
`VITE_API_BASE_URL`	`https://wealthwise-backend-zv5r.onrender.com`

4.4 render.yaml (Infrastructure as Code)

The project includes a `render.yaml` file that defines both services declaratively:

```
services:
  - type: web
    name: wealthwise-backend
    runtime: docker
    dockerfilePath: ./backend/Dockerfile
    dockerContext: ./backend
    plan: free
    healthCheckPath: /api/auth/health
    envVars:
      - key: SPRING_PROFILES_ACTIVE
        value: prod
      - key: PORT
        value: 8080

  - type: web
    name: wealthwise-frontend
    runtime: static
    staticPublishPath: ./project/dist
    buildCommand: cd project && npm ci && npm run build
    plan: free
    routes:
      - type: rewrite
        source: /*
        destination: /index.html
```

---

5. Cold-Start Warmup Architecture

Render's free tier spins down containers after 15 minutes of inactivity. WealthWise handles this with a multi-layer warmup strategy:

5.1 Frontend Warmup Overlay

On application load, `BackendWarmupContext` immediately pings `GET /api/auth/health`:

1. **If backend responds in <5s** → `isWarm = true` → no overlay shown
2. **If backend is cold** → Show animated overlay: "Waking up the server... Xs"
3. **Poll every 3s** until backend responds or 90s timeout
4. **After warm:** Keep-alive ping fires every 9 minutes to prevent re-spin-down

5.2 JVM Tuning for Fast Cold Start

```
-XX:TieredStopAtLevel=1   → Skip C2 JIT (saves ~20s startup)
-XX:MaxRAMPercentage=70.0 → Cap heap at 70% of 512 MB
-XX:+UseSerialGC          → Lower memory footprint than G1
-Xss512k                  → Smaller thread stacks
```

---

6. Build Commands Reference

ACTION	COMMAND	DIRECTORY
Backend — Install dependencies	`mvn dependency:resolve`	`backend/`
Backend — Build JAR	`mvn clean package -DskipTests`	`backend/`
Backend — Run (Maven)	`mvn spring-boot:run`	`backend/`
Backend — Run (JAR)	`java -jar target/wealthwise-backend-0.0.1-SNAPSHOT.jar`	`backend/`
Backend — Docker Build	`docker build -t wealthwise-backend .`	`backend/`
Backend — Docker Run	`docker run -p 8080:8080 --env-file .env.local wealthwise-backend`	`backend/`
Frontend — Install	`npm install`	`project/`
Frontend — Dev Server	`npm run dev`	`project/`
Frontend — Production Build	`npm run build`	`project/`
Frontend — Preview Build	`npm run preview`	`project/`
Frontend — Lint	`npm run lint`	`project/`
Scheme Seed	`curl -X POST http://localhost:8080/api/schemes/seed`	Any

---

7. Common Issues & Troubleshooting

Issue 1: Backend fails to start — "Failed to configure DataSource"

Cause: Database connection environment variables not set or incorrect.
Fix: Ensure `DB_URL`, `DB_USERNAME`, and `DB_PASSWORD` are set as system environment variables before running the app. Verify the Supabase connection string includes `?sslmode=require`.

Issue 2: "Could not resolve placeholder 'app.jwt.secret'"

Cause: `JWT_SECRET` environment variable not set.
Fix: Set `JWT_SECRET` with a ≥32-character string.

Issue 3: Frontend shows "Network Error" on all API calls

Cause: Backend not running or CORS not configured.
Fix:

1. Verify backend is running: `curl http://localhost:8080/api/auth/health`
2. Ensure `CORS_ALLOWED_ORIGINS` includes the frontend URL

Issue 4: CAS PDF import returns "Valid PDF file is required"

Cause: File is not a valid PDF or content-type header is wrong.
Fix: Ensure the uploaded file is a genuine PDF (not renamed).

Issue 5: "Duplicate key value violates unique constraint nav_history"

Cause: Concurrent NAV history insert race condition.
Fix: The system uses `INSERT ON CONFLICT DO NOTHING` — this should not occur. If it does, verify the nav_history unique constraint exists: `UNIQUE(amfi_code, nav_date)`.

Issue 6: Render backend takes >90 seconds to start

Cause: Free tier cold start with large dependency tree.
Fix: The Dockerfile uses multi-stage build with dependency caching. Ensure `mvn dependency:go-offline` layer is cached by not modifying `pom.xml` unnecessarily.

Issue 7: Gmail SMTP fails to send OTP

Cause: Gmail blocks "less secure app access."
Fix: Use a Gmail App Password instead of the regular password. Generate at:
https://myaccount.google.com/apppasswords

Issue 8: PAN card shows encrypted gibberish in database

Expected behavior. PAN cards are AES-256-GCM encrypted at rest. The decrypted value is only available through the API (masked as ABCDE****F).

Issue 9: Frontend build fails on Render

Cause: `VITE_API_BASE_URL` not set during build time.
Fix: Vite env vars must be set at build time (not just runtime). Set them in Render's Environment tab before triggering a deploy.

Issue 10: "Token invalid or expired" after restarting backend

Cause: `JWT_SECRET` changed between restarts.
Fix: Use a consistent `JWT_SECRET` across deployments. Changing it invalidates all existing tokens.

Issue 11: Scheme search returns no results

Cause: Scheme master database not seeded.
Fix: Run `POST /api/schemes/seed` to populate the database from AMFI NAVAll.txt.

Issue 12: Analytics shows "Insufficient data" for all metrics

Cause: No transactions recorded or NAV history not fetched.
Fix: Record at least one transaction. The analytics engine requires transaction data and fetches NAV history on demand from mfapi.in.

WealthWise — User Manual

1. System Overview

WealthWise is your personal mutual fund portfolio intelligence platform. It helps you:

- **Track** all your mutual fund investments in one place — across AMCs, folios, and platforms
- **Import** your complete investment history from CAMS/KFintech CAS PDF statements
- **Analyze** your portfolio with professional-grade analytics: returns (XIRR), risk profiling, and fund overlap detection
- **Optimize** your SIPs with intelligent recommendations on timing, step-up strategies, and comparison with lump-sum alternatives
- **Plan** your financial goals with Monte Carlo simulations that model 10,000 possible market scenarios
- **Secure** your data with bank-grade encryption and passwordless recovery via email OTP

2. Access & Installation

2.1 Production Access (Recommended)

WealthWise is deployed as a cloud application. No installation is needed.

1. Open your web browser (Chrome, Firefox, Edge, or Safari)
2. Navigate to: **https://wealthwise-frontend-8xmb.onrender.com**
3. If you see a "Waking up the server" overlay, please wait 30–90 seconds — the backend is starting up from idle

2.2 Local Access (For Development)

If running locally:

1. Start the backend: Open terminal in `backend/` → Run `mvn spring-boot:run`
2. Start the frontend: Open terminal in `project/` → Run `npm run dev`
3. Open: **http://localhost:5173**

3. Login & Registration

3.1 Creating a New Account (Sign Up)

1. Click **"Get Started"** on the landing page hero section, or **"Sign Up"** in the navigation bar
2. The authentication modal opens on the **Sign Up** tab
3. Fill in the registration form:

FIELD	REQUIRED	DESCRIPTION
Full Name	✅	Your legal name as per PAN card
Email Address	✅	Used for login and OTP-based password recovery
Password	✅	Minimum 8 characters; choose a strong password
Phone Number	❌	Mobile number with country code
Currency	❌	Display currency preference (default: INR)
PAN Card	❌	10-character PAN number; stored encrypted (AES-256)

4. Click **"Create Account"**

5. On success, you are automatically logged in and redirected to the Dashboard

3.2 Signing In

1. Click **"Sign In"** in the navigation bar
2. Enter your registered email and password
3. Click **"Sign In"**
4. On success, you are redirected to the Dashboard

3.3 Forgot Password

If you forget your password:

1. Click **"Forgot Password?"** on the Sign In form
2. Enter your registered email address → Click **"Send OTP"**
3. Check your email inbox for a 6-digit OTP (arrives within 30 seconds)
4. Enter the OTP and your new password (minimum 8 characters)
5. Click **"Reset Password"**
6. You can now sign in with your new password

> ***Note:*** *The OTP expires after 5 minutes. If it expires, click "Send OTP" again.*

3.4 Signing Out

Click your profile icon in the navigation bar → Click **"Sign Out"**. Your session is cleared from the browser.

---

4. Feature-by-Feature Guide

4.1 Dashboard

Access: Click "Dashboard" in the navigation bar (or you are redirected here after login)
Purpose: Your portfolio headquarters — a real-time overview of all investments

The Dashboard is divided into the following sections:

4.1.1 Portfolio Value Card

- **Total Invested:** The sum of all your purchase amounts (lump sums + SIPs)
- **Current Value:** Your portfolio valued at today's NAV prices
- **Unrealized P&L:** The difference (Current Value – Total Invested), shown in green (profit) or red (loss)
- **Overall Return %:** Percentage return on your total investment

4.1.2 Portfolio Allocation Chart

- A pie/donut chart showing how your money is distributed across:
  - **Categories:** Equity, Debt, Hybrid, Solution-Oriented
  - **Schemes:** Individual fund allocations

4.1.3 Holdings Table

- A detailed table listing every scheme in your portfolio:
  - Scheme Name
  - AMFI Code
  - Units Held
  - Purchase NAV (average)
  - Current NAV
  - Invested Amount

- Current Value
- P&L (amount and percentage)

4.1.4 Growth Timeline Chart

- An area chart showing how your portfolio value has changed over time
- X-axis: months; Y-axis: portfolio value in INR
- Useful for visualizing the trajectory of your wealth

4.1.5 Quick Actions

- **Add Transaction** — Navigate to Transactions page
- **Import CAS** — Upload a CAS PDF
- **View Analytics** — Navigate to the Analytics page

4.2 Transactions

Access: Click "Transactions" in the navigation bar
Purpose: Record, import, and manage all mutual fund transactions

4.2.1 Recording a Manual Transaction

1. Click **"Add Transaction"** button
2. Fill in the transaction form:

FIELD	DESCRIPTION
Scheme	Start typing the scheme name — a live search dropdown appears. Select your fund.
Transaction Type	Choose: Lumpsum Purchase, SIP, Redemption, Switch In/Out, SWP, STP, Dividend
Date	The date of the transaction
Amount	The invested/redeemed amount in INR
NAV	Auto-fetched for the selected date. You can override manually.
Folio Number	Your AMC folio number (format: 12345678/01)
Notes	Optional free-text notes

3. Click **"Record Transaction"**

> ***Tip:*** *When you select a date, the system automatically fetches the NAV for that date from AMFI data. If the date is a market holiday, you'll see a message to use the nearest available date.*

4.2.2 Bulk SIP Generator

For importing historical SIP data in bulk:

1. Click **"Bulk SIP"**
2. Select the scheme, enter the monthly amount
3. Set the start date and end date (maximum 10 years)
4. Click **"Generate"** — the system creates one transaction per month with the historical NAV for each date

4.2.3 CAS PDF Import

For importing your complete transaction history from CAMS or KFintech:

1. Click **"Import CAS"**
2. Select your CAS PDF file (maximum 10 MB)
3. The system parses all folios, schemes, and transactions automatically
4. A summary appears showing: number of folios imported, number of transactions, and any schemes that couldn't be matched

> ***Tip:*** *Download your latest CAS statement from https://www.camsonline.com or https://kfintech.com*

4.2.4 Transaction Reversal

If you recorded a transaction by mistake:

1. Find the transaction in your transaction list
2. Click the **"Reverse"** button on that transaction
3. A mirror reversal transaction is created that undoes the original (returns units to your lot)

4.2.5 Transaction Filters

- Filter by scheme, transaction type, date range, or source (Manual / CAS Import)
- Transactions are sorted by date (newest first)

4.2.6 Portfolio Summary View

- The Portfolio Summary section at the top of the Transactions page shows a per-scheme aggregation:
  - Total units, average cost, current value, and P&L for each scheme

---

4.3 Analytics

Access: Click "Analytics" in the navigation bar
Purpose: Deep portfolio analysis with institutional-grade metrics

4.3.1 Risk Profile

The Risk Profile section displays:

METRIC	DESCRIPTION	HOW TO READ
Portfolio Volatility	Annualized standard deviation of daily NAV returns	<15% = Low, 15-25% = Moderate, >25% = High
Sharpe Ratio	Risk-adjusted return (excess return per unit of risk)	>1.0 = Good, >1.5 = Excellent, <0.5 = Poor
Risk Score	SEBI riskometer score (1-6) weighted by portfolio allocation	1-2 = Conservative, 3-4 = Moderate, 5-6 = Aggressive
Risk Category Badge	Your overall portfolio risk level	CONSERVATIVE / MODERATE / AGGRESSIVE

Each scheme also shows its individual risk metrics for comparison.

4.3.2 Fund Overlap Analysis

The Overlap section shows:

- **Overlap Matrix:** A heatmap grid showing the percentage of stock overlap between every pair of funds in your portfolio
- **High Overlap Pairs:** Pairs with >30% overlap are highlighted in red with a warning
- **Common Stocks:** Click on a pair to see which specific stocks are held by both funds
- **Consolidation Suggestions:** Actionable recommendations like "Axis Bluechip and SBI Bluechip have 68% overlap — consider consolidating into one"

> ***Why does overlap matter?*** *If two funds hold the same stocks, you're paying two expense ratios for the same exposure. Consolidating reduces costs and simplifies your portfolio.*

4.3.3 SIP Intelligence

The SIP section provides:

- **Active SIPs Count:** Number of SIPs detected as currently active
- **Monthly Outflow:** Total monthly SIP commitment

- **SIP Streak:** How many consecutive months you've maintained all SIPs
- **Health Alerts:** Warnings if SIPs appear to have stopped or are irregular

4.3.4 Per-Scheme Risk Breakdown

A table showing volatility, Sharpe ratio, and risk level for each individual scheme, alongside its allocation percentage in your portfolio.

4.4 SIP Intelligence (within Analytics)

4.4.1 SIP Dashboard

Shows a card with your SIP health: total SIPs, monthly outflow, streak, and projected corpus at current rates.

4.4.2 SIP vs. Lumpsum Comparison

Answers the question: "Would I have been better off investing the same total amount as a lump sum on day one?"

- Shows both values side by side
- Declares a winner with the difference amount
- In rising markets, lumpsum typically wins; in volatile markets, SIP wins via rupee-cost averaging

4.4.3 SIP Day Optimizer

Analyzes historical NAV patterns to find the day of the month when NAV tends to be lowest:

- Recommends switching your SIP date
- Shows potential savings percentage

4.4.4 Step-Up Projection

Models two scenarios over 10 years:

- **Without Step-Up:** Your current SIP amount stays the same
- **With 10% Step-Up:** Your SIP increases by 10% annually
- Shows the difference in final corpus (often 50-80% more with step-up)

4.5 Goals

Access: Click "Goals" in the navigation bar
Purpose: Create financial goals and test their feasibility with scientific modeling

4.5.1 Creating a Goal

1. Click **"Create New Goal"**
2. The Goal Wizard walks you through:

STEP	FIELDS
Step 1: Basic Info	Goal name (e.g., "Retirement Fund"), Goal type (Retirement, House, Education, Wedding, etc.), Icon
Step 2: Target	Target amount in today's money (e.g., ₹5 Crore), Expected inflation rate (default: 6%), Target date
Step 3: Investment	Monthly SIP allocation, Expected return rate (default: 12% pa), Priority (Low/Medium/High)

3. Click **"Save Goal"**

4.5.2 Analyzing a Goal

1. On the Goals page, click **"Analyze"** on any goal card
2. The Analysis Modal opens with three tabs:

Tab 1: Monte Carlo Simulation

- Runs 10,000 random market scenarios
- Shows three outcome ranges:

- **Pessimistic (P10):** Only 10% of scenarios were worse than this
- **Likely (P50):** The median (most probable) outcome
- **Optimistic (P90):** Only 10% of scenarios were better than this

- Shows **Probability of Success:** the percentage of scenarios where you reached your goal
- All values shown in **today's money** (inflation-adjusted)

Tab 2: Deterministic Projection

- Shows a simple compound growth projection
- Includes sensitivity analysis:
  - "What if returns are only 10%?" — shows the reduced corpus
  - "What if you miss 6 SIPs?" — shows the impact of inconsistency
  - "What if inflation is 2% higher?" — shows the real purchasing power erosion

Tab 3: Required SIP Calculator

- Tells you exactly how much monthly SIP is needed to reach your goal
- Shows the gap between your current SIP and the required SIP
- Provides alternatives: a one-time lump sum amount, or additional months needed at current SIP rate

4.5.3 Linking Funds to Goals

1. On a goal card, click **"Link Funds"**
2. Select investment lots from your portfolio
3. Set allocation percentages (e.g., 50% of your Axis Bluechip lot goes toward this goal)
4. The goal tracker updates to show how much is already funded

---

4.6 Profile

Access: Click your profile icon in the navigation bar → "Profile"
Purpose: Manage your personal information and security settings

4.6.1 View Profile

- See your full name, email, phone, currency, join date, and risk profile
- PAN card is displayed in masked format: **ABCDE****F

4.6.2 Update Profile

- Edit your full name, phone number, currency, and PAN card
- Click **"Save Changes"**

4.6.3 Change Password

- Enter your current password and new password
- New password must be at least 8 characters
- Click **"Change Password"**

4.6.4 Risk Profile Setting

- Set your preferred risk profile: Conservative, Moderate, or Aggressive
- This helps the analytics engine compare your actual portfolio risk against your preference

---

4.7 Market Section (Landing Page)

The landing page includes a **Market Section** that displays:

- Real-time market indices (Nifty 50, Sensex)
- Trending mutual fund schemes
- Top-performing schemes by category

This section is visible to all visitors (no login required) and serves as a discovery tool.

5. Navigation Guide

5.1 Navigation Bar

The navigation bar adapts based on your login state:

When NOT logged in:

ITEM	ACTION
WealthWise logo	Navigate to landing page
Features	Scroll to features section
Analytics	Scroll to analytics section
Sign In	Open authentication modal (Sign In tab)
Get Started	Open authentication modal (Sign Up tab)
🌙 / ☀️	Toggle dark/light theme

When logged in:

ITEM	ACTION
WealthWise logo	Navigate to dashboard
Dashboard	Navigate to portfolio dashboard
Transactions	Navigate to transaction management
Analytics	Navigate to analytics suite
Goals	Navigate to goal planner
Profile icon	Open profile dropdown (Profile, Sign Out)
🌙 / ☀️	Toggle dark/light theme

5.2 Protected Routes

The following pages require authentication:

- `/dashboard` → Portfolio Dashboard
- `/transactions` → Transaction Management
- `/analytics` → Analytics Suite
- `/goals` → Goal Planner
- `/profile` → User Profile

If you try to access these pages without being logged in, you are automatically redirected to the landing page.

If your session expires (JWT token expiry), you are immediately signed out and redirected.

6. Screen Descriptions

6.1 Landing Page (`/`)

A premium dark-mode landing page with animated particle background consisting of:

1. **Hero Section** — Large headline "The Future of Portfolio Intelligence," subtext describing the platform, and two CTAs: "Get Started" and "Explore Features"
2. **Mutual Fund Section** — Live market data cards and trending scheme tickers
3. **Analytics Section** — Visual showcase of portfolio analytics capabilities with animated charts

4. **Features Grid** — 6 feature cards with icons: Portfolio Tracking, CAS Import, Risk Analytics, SIP Intelligence, Goal Planning, Secure Vault
5. **CTA Section** — "Start Your Wealth Journey Today" with Sign Up button
6. **Footer** — Links, copyright, and social media icons

6.2 Dashboard Page (`/dashboard`)

A data-dense financial dashboard with dark glassmorphism cards:

- **Top Row:** Portfolio value tile (invested / current / P&L), category allocation donut chart
- **Middle Row:** Holdings table with sortable columns and search
- **Bottom Row:** Growth timeline area chart, quick action buttons

6.3 Transactions Page (`/transactions`)

A tabular interface for transaction management:

- **Top Bar:** "Add Transaction" and "Import CAS" buttons, filter controls
- **Transaction Form Modal:** Scheme search input, type selector, date picker, amount/NAV fields
- **Portfolio Summary Strip:** Per-scheme ribbons showing units, value, and return
- **Transaction List:** Sortable, filterable table with all transactions; each row has a "Reverse" button

6.4 Analytics Page (`/analytics`)

A multi-panel analytics dashboard:

- **Risk Profile Panel:** Volatility gauge, Sharpe ratio badge, risk score ring
- **Fund Overlap Panel:** Interactive heatmap matrix with click-to-expand stock lists
- **SIP Intelligence Panel:** SIP summary cards, comparison chart, optimization recommendation
- **Per-Scheme Risk Table:** Detailed risk breakdown per fund

6.5 Goals Page (`/goals`)

A card-based goal management interface:

- **Goal Cards:** Each goal shows: name, icon, target amount, progress bar, target date, priority badge
- **Goal Wizard Modal:** Step-by-step form for creating new goals
- **Analysis Modal:** Three-tab interface showing Monte Carlo results, deterministic projections, and required SIP calculations with charts

6.6 Profile Page (`/profile`)

A forms-based profile management page:

- **Personal Information Card:** Editable fields for name, phone, currency, PAN
- **Security Card:** Change password form with current/new password fields
- **Risk Profile Card:** Three selectable badges (Conservative/Moderate/Aggressive)
- **Account Info Card:** Read-only display of email, join date, and account ID

6.7 Warmup Overlay

A full-screen overlay that appears when the backend is cold-starting:

- Animated pulsing circle
- "Waking up the server..." message
- Elapsed time counter (e.g., "25s")
- Progress indication
- Auto-dismisses when backend responds to health check

---

7. Frequently Asked Questions (FAQ)

Q1: Is my financial data secure?

Yes. Passwords are hashed with BCrypt (industry standard). PAN card numbers are encrypted with AES-256-GCM (military-grade encryption). All communication uses HTTPS. Your data is never shared with third parties.

Q2: Why does the app take 30-90 seconds to load sometimes?

WealthWise is hosted on Render's free tier, which puts the server to sleep after 15 minutes of inactivity. The first visitor wakes it up, which takes 30-90 seconds. After that, it responds instantly for all users.

Q3: How accurate are the NAV values?

NAV data is sourced from mfapi.in, which pulls directly from AMFI's official database. Values are accurate to 4 decimal places and cached for 24 hours.

Q4: Can I import my entire mutual fund history?

Yes! Download your CAS (Consolidated Account Statement) from CAMS (www.camsonline.com) or KFintech, and upload the PDF on the Transactions page. WealthWise parses all folios, schemes, and transactions automatically.

Q5: What is XIRR and why should I care?

XIRR (Extended Internal Rate of Return) is the most accurate way to measure investment returns when you have multiple investments at different times (like monthly SIPs). Unlike simple return, XIRR accounts for the timing of each cash flow, giving you a true annualized return figure.

Q6: What does "Fund Overlap" mean?

If you hold multiple mutual funds, they may be investing in the same underlying stocks. For example, two large-cap funds might both hold Reliance, HDFC Bank, and TCS. This "overlap" means you're paying two expense ratios for the same exposure — reducing diversification.

Q7: How does the Monte Carlo simulation work?

The Monte Carlo engine simulates your portfolio 10,000 times with random monthly returns drawn from a normal distribution. This models the inherent uncertainty of markets and gives you a range of possible outcomes (pessimistic to optimistic) rather than a single unrealistic projection.

Q8: Can I use WealthWise without importing CAS?

Yes! You can manually enter transactions one by one, or use the Bulk SIP generator to quickly enter historical SIP data.

Q9: What transaction types are supported?

Lumpsum Purchase, SIP, Redemption, Switch In/Out, Systematic Withdrawal (SWP), Systematic Transfer (STP In/Out), Dividend Payout, Dividend Reinvestment, and Reversal.

Q10: Is the ELSS lock-in period tracked?

Yes. When you record a transaction for an ELSS scheme, the system automatically sets a 3-year lock-in period. Redemptions within the lock-in period are blocked.

Q11: How often is scheme data updated?

The scheme master database can be refreshed on demand from AMFI's NAVAll.txt file, which contains 45,000+ schemes. This is typically done by the system administrator.

Q12: Can I delete my account?

Currently, account deletion is not self-serve. Contact the administrator to request account deletion along with all associated data.

Q13: What happens to my data if the server restarts?

All data is stored in PostgreSQL (hosted on Supabase with automated backups). NAV data is persisted as a write-through cache, so nothing is lost on server restarts.

Q14: Can I access WealthWise on my phone?

WealthWise is a responsive web application that works on tablet-sized screens and above. A dedicated mobile app is not currently available, but you can access it via your phone's browser.

Q15: What if my CAS PDF has schemes that can't be identified?

The system generates temporary internal codes for unrecognized schemes. You can trigger a "Reconciliation" to automatically match these codes against the AMFI database. In most cases, 90%+ of schemes are resolved automatically.

Testing Documentation

***Document No.**: 07 of 07*
***Project**: WealthWise — Indian Mutual Fund Portfolio Intelligence Platform*
***Stack**: Spring Boot 3.2.3 · Java 17 · React 19 · Vite · PostgreSQL (Supabase)*
***Date**: April 2026*
***Author**: Bhuvan Nagesh | VTU Internship 2026 — Team 11*

Test Results Summary

LAYER	TESTS	PASS	FAIL	STATUS
Backend (JUnit 5 / Maven)	235	235	0	BUILD SUCCESS
Frontend (Vitest)	45	45	0	All Passing
Grand Total (Automated)	**280**	**280**	**0**	**100% Pass Rate**

```
Backend build output:
[INFO] Tests run: 235, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS — Total time: 22.4 s

Frontend build output:
  src/test/components.test.jsx   (30 tests) 214ms
  src/test/integration.test.js   (15 tests) 177ms
  Tests  45 passed (45) | Duration: 2.46s
```

Requirements Coverage Matrix

#	REQUIREMENT	TOOL / APPROACH	STATUS
1	Frontend Unit Testing — utility functions, hooks, components	Vitest + React Testing Library	Done
2	Frontend Integration Testing — API calls with mock backend	Vitest + Mock Service Worker (MSW)	Done
3	Visual Regression — charts and animations	Storybook + Chromatic	Done
4	Backend Unit Testing — services, utilities, isolated logic	JUnit 5 + Mockito	Done
5	Backend PDFBox unit testing — CAS PDF parser	JUnit 5 + Java Reflection	Done
6	Integration Testing — full stack with real database	@SpringBootTest + Testcontainers	Done
7	Security Validation — JWT, @WithMockUser, RBAC	Spring Security Test	Done
8	Cache Testing — Caffeine TTL, hit/miss, eviction	CaffeineCacheManager unit tests	Done
9	Database Schema and Constraint Validation	PostgreSQL SQL scripts	Done
10	Data Isolation (Row-Level Security)	PL/pgSQL isolation test	Done
11	End-to-End Testing — Login, PDF upload, Dashboard	Playwright (script ready)	Done
12	Performance and Load Testing — breaking point	k6 (3 scenarios ready)	Done
13	SAST — Static Analysis	SonarQube (configured)	Done
14	Dependency Scanning — CVE detection	OWASP Dependency-Check (pom.xml)	Done

Test File Inventory

Backend Test Files

```
backend/src/test/java/com/wealthwise/
├── config/
│   └── TestSecurityConfig.java           Helper — permit-all for @WebMvcTest
├── controller/
│   ├── AuthControllerTest.java           TS-AUTH-001       18 tests
│   ├── AnalyticsControllerTest.java      TS-ANA-001        18 tests
│   └── GoalEngineControllerTest.java     TS-GOAL-001       14 tests
├── integration/
│   └── WealthWiseIntegrationTest.java    TS-TC-001         12 tests (Docker required)
├── repository/
│   └── RepositoryLayerTest.java          TS-REPO-001       15 tests
├── security/
│   ├── JwtServiceTest.java               TS-JWT-001        15 tests
│   ├── JwtAuthenticationFilterTest.java  TS-SEC-001        15 tests
│   └── SpringSecurityValidationTest.java TS-SPRINGSEC-001  10 tests
└── service/
    ├── GoalEngineServiceTest.java        TS-GES-001        30 tests
    ├── GoalEngineBoundaryTest.java       TS-BOUND-001      45 tests
    ├── AuthServiceTest.java              TS-AUTH-SVC-001   18 tests
    ├── CasPdfParserServiceTest.java      TS-PDF-001        28 tests
    └── NavServiceCacheTest.java          TS-CACHE-001       9 tests
```

Frontend Test Files

```
project/src/
├── test/
│   ├── setup.js                          jest-dom global setup
│   ├── components.test.jsx               TS-FE-001         30 tests
│   ├── integration.test.js               TS-INT-FE-001     15 tests
│   └── e2e/
│       └── wealthwise.spec.js            TS-E2E-001        15 cases (Playwright)
└── stories/
    ├── ErrorBoundary.stories.jsx         TS-VR-001          2 story states
    ├── Stats.stories.jsx                 TS-VR-002          3 story states
    └── WarmupOverlay.stories.jsx         TS-VR-003          3 story states
```

Suite 1 — GoalEngineService Unit Tests (TS-GES-001)

File: `GoalEngineServiceTest.java` | **Type:** Pure unit, JUnit 5 + Mockito | **Tests:** 30

TC ID	TEST NAME	WHAT IS VERIFIED
TC-GES-001	monteCarlo_probabilityBetween0And1	Probability result is in range [0.0, 1.0]
TC-GES-002	monteCarlo_percentileOrderP10LtP50LtP90	P10 <= P50 <= P90 always holds
TC-GES-003	monteCarlo_zeroSipLowerOutcomes	Zero SIP produces lower P90 than positive SIP
TC-GES-004	monteCarlo_highVolatilityWiderSpread	High volatility widens the P10 to P90 gap
TC-GES-005	monteCarlo_negativeMeanStillComputes	Negative monthly mean does not crash the engine
TC-GES-006	monteCarlo_targetExceeded_probabilityHigh	Very high target gives low probability

TC ID	TEST NAME	WHAT IS VERIFIED
TC-GES-007	monteCarlo_targetZero_probability1	Target of 0 means probability must equal 1.0
TC-GES-008	monteCarlo_singleMonth_returnsReasonableValue	months=1 produces a non-negative result
TC-GES-009	monteCarlo_largePortfolio_highProbability	Large initial portfolio produces high success probability
TC-GES-010	monteCarlo_inflationAdjusted_lowerProb	Inflation lowers real purchasing power correctly
TC-GES-011	deterministic_onTrackTrue_whenProjectedExceedsTarget	FV > target means onTrack = true
TC-GES-012	deterministic_onTrackFalse_whenProjectedBelowTarget	FV < target means onTrack = false
TC-GES-013	deterministic_gapSign_consistentWithOnTrack	gap <= 0 if and only if onTrack = true
TC-GES-014	deterministic_sensitivityTableHas3Scenarios	sensitivity list always has exactly 3 entries
TC-GES-015	deterministic_fvComponents_nonNegative	fvCorpus >= 0 and fvSip >= 0
TC-GES-016	deterministic_zeroContribution_onlyFvCorpus	Zero SIP means fvSip = 0
TC-GES-017	deterministic_zeroInitial_onlyFvSip	Zero initial amount means fvCorpus = 0
TC-GES-018	deterministic_inflationAdjustsTarget	Inflation increases the real target amount
TC-GES-019	deterministic_totalProjected_equalsSum	totalProjected = fvCorpus + fvSip always
TC-GES-020	deterministic_highReturn_onTrack	20% yearly return puts moderate target on track
TC-GES-021	requiredSip_sipGapInvariant	requiredSip = currentSip + sipGap always
TC-GES-022	requiredSip_whenCurrentSipEnough	currentSipEnough = true when sip >= required
TC-GES-023	requiredSip_lumpSumToday_nonNegative	lumpSumToday is always >= 0
TC-GES-024	requiredSip_extraMonths_positiveWhenInsufficient	extraMonths > 0 when SIP is insufficient
TC-GES-025	requiredSip_zeroCurrentSip_largeGap	Zero currentSip means sipGap = requiredSip
TC-GES-026	requiredSip_veryHighTarget_largeRequired	10 Crore target produces a large required SIP
TC-GES-027	requiredSip_highInitialPortfolio_lowRequired	Large initial corpus reduces the required SIP
TC-GES-028	requiredSip_shortHorizon_highRequired	Short horizon means higher required SIP
TC-GES-029	requiredSip_longHorizon_lowRequired	Long horizon means lower required SIP

TC ID	TEST NAME	WHAT IS VERIFIED
TC-GES-030	requiredSip_requiredAlwaysPositive	requiredSip > 0 for any positive target

---

Suite 2 — GoalEngine Boundary and Parameterized Tests (TS-BOUND-001)

File: `GoalEngineBoundaryTest.java` | **Type**: @ParameterizedTest, JUnit 5 | **Tests**: 45

GROUP	INPUT DIMENSION	VALUES TESTED	TEST COUNT
MonteCarlo Horizons	months	1, 6, 12, 60, 120, 360	6
MonteCarlo Volatility	stdDev	0.02, 0.05, 0.10, 0.15, 0.20	5
MonteCarlo Inflation	inflation + target	(2%, 1M), (6%, 2M), (12%, 5M)	3
Deterministic Horizons	months	12, 24, 36, 60, 84, 120	6
Deterministic Returns	annualReturn	6%, 8%, 12%, 15%	4
Deterministic Targets	targetAmount	1L, 5L, 50L, 1Cr	4
RequiredSIP Levels	currentSip	0, 1000, 5000, 15000, 50000	5
Single assertions	Edge cases	Invariant checks	12

Mathematical invariants validated across all parameter combinations: P10 <= P50 <= P90 (percentile monotonicity), requiredSip = currentSip + sipGap (SIP gap invariant), gap <= 0 if and only if onTrack = true (sign consistency), totalProjected = fvCorpus + fvSip (component decomposition).

Suite 3 — JwtService Unit Tests (TS-JWT-001)

File: `JwtServiceTest.java` | **Type**: Pure unit, real HMAC-SHA256, no mocks | **Tests**: 15

TC ID	WHAT IS VERIFIED
TC-JWT-001	Token is never null
TC-JWT-002	Token is never an empty string
TC-JWT-003	JWT has three dot-separated segments (header.payload.signature)
TC-JWT-004	Two consecutive calls produce different tokens
TC-JWT-005	Different emails produce different tokens
TC-JWT-006	Email extracted from token matches what was encoded
TC-JWT-007	Exact email value is preserved in the payload
TC-JWT-008	Special characters in email such as plus, dot, and hyphen are handled
TC-JWT-009	Expiry timestamp is always in the future
TC-JWT-010	Expiry approximates now plus the configured TTL (within one minute)
TC-JWT-011	Valid token passes validation
TC-JWT-012	Modified signature is rejected
TC-JWT-013	Expired token is rejected
TC-JWT-014	Token signed with a different secret is rejected
TC-JWT-015	Token with a mismatched email is rejected

Suite 4 — AuthService Unit Tests (TS-AUTH-SVC-001)

File: `AuthServiceTest.java` | **Type:** Unit, Mockito mocks | **Tests:** 18

TC ID	GROUP	WHAT IS VERIFIED
TC-AUTH-SVC-001	Registration	BCrypt encoder is called before saving the user
TC-AUTH-SVC-002	Registration	repository.save() is called exactly once
TC-AUTH-SVC-003	Registration	Returned user object is not null
TC-AUTH-SVC-004	Registration	Stored password is not equal to the original plaintext
TC-AUTH-SVC-005	Authentication	Correct email and password returns the user
TC-AUTH-SVC-006	Authentication	Unknown email throws RuntimeException
TC-AUTH-SVC-007	Authentication	Wrong password throws RuntimeException
TC-AUTH-SVC-008	Authentication	PasswordEncoder.matches() is used, not plain string equals
TC-AUTH-SVC-009	OTP Generation	Generated OTP matches the regex pattern for 6 digits
TC-AUTH-SVC-010	OTP Generation	PasswordEncoder.encode() is called on the OTP before storage
TC-AUTH-SVC-011	OTP Generation	OTP expiry is set to now plus 5 minutes
TC-AUTH-SVC-012	OTP Generation	EmailService.sendOtpEmail() is called exactly once
TC-AUTH-SVC-013	OTP Generation	No email is sent for an unknown email address
TC-AUTH-SVC-014	Password Reset	Correct OTP within expiry successfully resets the password
TC-AUTH-SVC-015	Password Reset	Wrong OTP throws RuntimeException
TC-AUTH-SVC-016	Password Reset	Expired OTP throws RuntimeException with expired in the message
TC-AUTH-SVC-017	Password Reset	resetOtp and otpExpiry are set to null after successful reset
TC-AUTH-SVC-018	Password Reset	New password is encoded before saving

Suite 5 — CasPdfParserService Unit Tests — Apache PDFBox (TS-PDF-001)

File: `CasPdfParserServiceTest.java` | **Type:** Unit via Java Reflection | **Tests:** 28

Strategy: Uses Method.setAccessible(true) to invoke private helper methods directly without loading any PDF file. Zero disk I/O required.

detectType() — Transaction Classifier (13 tests)

TC ID	INPUT TEXT	EXPECTED TRANSACTION TYPE
TC-PDF-001	"SIP Purchase"	PURCHASE_SIP
TC-PDF-002	"Systematic Investment Plan"	PURCHASE_SIP
TC-PDF-003	"SWITCH IN Transfer"	SWITCH_IN
TC-PDF-004	"SWITCH OUT Transfer"	SWITCH_OUT
TC-PDF-005	"Redemption Payout"	REDEMPTION
TC-PDF-006	"STP In Transfer"	STP_IN
TC-PDF-007	"STP Out Transfer"	STP_OUT
TC-PDF-008	"Dividend Payout IDCw"	DIVIDEND_PAYOUT

TC ID	INPUT TEXT	EXPECTED TRANSACTION TYPE
TC-PDF-009	"SWP Monthly Withdrawal"	SWP
TC-PDF-010	"Bonus Units"	BONUS
TC-PDF-011	"IDCW Reinvestment"	DIVIDEND_REINVEST
TC-PDF-012	null	PURCHASE_LUMPSUM (safe default)
TC-PDF-013	"Unknown XYZ"	PURCHASE_LUMPSUM (fallback)

parseBigDecimal() — Indian Number Format Parser (7 tests)

TC ID	INPUT	EXPECTED OUTPUT
TC-PDF-014	"1,87,432.56"	BigDecimal 187432.56
TC-PDF-015	"5000.00"	BigDecimal 5000.00
TC-PDF-016	"1,00,00,000.00"	BigDecimal 10000000.00
TC-PDF-017	null	null
TC-PDF-018	blank string	null
TC-PDF-019	"ABC"	null
TC-PDF-020	"(1,500.00)"	null or handled safely

extractSearchKeyword() — Keyword Extractor (8 tests)

TC ID	INPUT	EXPECTED OUTPUT
TC-PDF-021	"Axis Bluechip Fund"	"Axis Bluechip"
TC-PDF-022	"Mirae Asset Large Cap Fund - Direct Plan"	"Mirae Asset"
TC-PDF-023	"ICICI Prudential - Regular Plan"	"ICICI Prudential"
TC-PDF-024	"SBI Bluechip Fund-Direct Growth"	non-null result
TC-PDF-025	null	null (safe)
TC-PDF-026	"OneWord"	"OneWord"
TC-PDF-027	"HDFC ELSS Tax Saver Fund IDCW"	IDCW suffix stripped
TC-PDF-028	"Nippon India"	"Nippon India"

Suite 6 — Caffeine Cache Tests (TS-CACHE-001)

File: `NavServiceCacheTest.java` | **Type**: Pure library unit tests | **Tests**: 9

NavService uses @Cacheable with a 24-hour TTL for nav_latest and a 7-day TTL for nav_history. These tests validate the Caffeine cache library contract that the annotations rely on.

TC ID	GROUP	WHAT IS VERIFIED
TC-CACHE-001	Cache Lifecycle	put() followed by get() round-trip preserves the stored value
TC-CACHE-002	Cache Lifecycle	get() returns null for a key that was never stored
TC-CACHE-003	Cache Lifecycle	Two different amfiCodes have entirely separate cache entries
TC-CACHE-004	Cache Lifecycle	evict() removes the entry and the subsequent get() returns null
TC-CACHE-005	Cache Lifecycle	clear() removes all entries simultaneously

TC ID	GROUP	WHAT IS VERIFIED
TC-CACHE-006	Named Cache	nav_latest and nav_history are separate namespaces
TC-CACHE-007	Named Cache	Clearing nav_latest does not remove any nav_history entries
TC-CACHE-008	TTL Expiry	Entry is accessible immediately after put within a 5-second TTL
TC-CACHE-009	TTL Expiry	Entry is null after the 1ms TTL expires (50ms sleep plus cleanUp())

Suites 7, 8, 9 — Web Layer Tests (TS-AUTH-001, TS-ANA-001, TS-GOAL-001)

Type: @WebMvcTest + TestSecurityConfig + MockMvc | **Total:** 50 tests

AuthController (18 tests)

GROUP	TESTS	WHAT IS COVERED
Health	2	GET /api/auth/health returns 200 with "Backend is running"
Sign Up	6	Valid signup returns 200 and token; invalid inputs return 400
Sign In	6	Valid login returns JWT; wrong password returns 401; missing fields return 400
Password Reset	4	Send OTP (200), verify OTP (200), wrong OTP (400), expired OTP (400)

AnalyticsController (18 tests)

GROUP	TESTS	WHAT IS COVERED
Risk Profile GET	5	Returns riskProfile, volatility, sharpeRatio, overallHealthScore
SIP Intelligence	4	Returns monthlyInvestment, projectedCorpus, XIRR, consistency score
Overlap	4	Returns overlapMatrix; empty portfolio returns 200 not 500
Risk Profile PATCH	5	Accepts AGGRESSIVE and CONSERVATIVE; rejects invalid values with 400

GoalEngineController (14 tests)

GROUP	TESTS	WHAT IS COVERED
Happy Path	6	POST to montecarlo, deterministic, requiredsip return 200 with correct fields
Validation	8	Missing initialPortfolio (400), negative months (400), null target (400), negative SIP (400)

Suite 10 — Repository Layer Tests (TS-REPO-001)

File: `RepositoryLayerTest.java` | **Type:** @DataJpaTest + H2 + real Hibernate DDL | **Tests:** 15

TC ID	GROUP	WHAT IS VERIFIED
TC-REPO-001	User	save() generates a positive auto-increment ID
TC-REPO-002	User	findByEmail finds an existing user
TC-REPO-003	User	findByEmail returns an empty Optional for an unknown email
TC-REPO-004	User	existsByEmail returns true for an existing email
TC-REPO-005	User	New user default riskProfile is MODERATE
TC-REPO-006	Transaction	Paginated query returns the correct page

TC ID	GROUP	WHAT IS VERIFIED
TC-REPO-007	Transaction	User A transactions are not returned when querying User B
TC-REPO-008	Transaction	Results are ordered by transaction date descending
TC-REPO-009	Transaction	Distinct scheme codes are correctly deduplicated
TC-REPO-010	Transaction	Date range filter returns only matching transactions
TC-REPO-011	Transaction	JPQL bulk update returns the affected row count
TC-REPO-012	Transaction	All transactions for a scheme code are returned
TC-REPO-013	Transaction	New transaction default source is MANUAL
TC-REPO-014	Transaction	Distinct folio numbers are returned correctly
TC-REPO-015	Transaction	Duplicate transactionRef is rejected by the database unique constraint

---

Suite 11 — JwtAuthenticationFilter Tests (TS-SEC-001)

File: `JwtAuthenticationFilterTest.java` | **Type:** Unit, MockHttpServletRequest | **Tests:** 15

TC ID	GROUP	WHAT IS VERIFIED
TC-SEC-001	Public Paths	/api/auth/ passes through without a JWT
TC-SEC-002	Public Paths	/api/schemes/ is publicly accessible
TC-SEC-003	Public Paths	/api/nav/ is publicly accessible
TC-SEC-004	Public Paths	/api/analytics/ requires a valid JWT
TC-SEC-005	Missing Token	Missing Authorization header returns 401
TC-SEC-006	Missing Token	Wrong scheme prefix such as Basic returns 401
TC-SEC-007	Missing Token	Bearer with no token value returns 401
TC-SEC-008	Missing Token	Filter chain is not called when 401 is returned
TC-SEC-009	Valid Token	userId attribute is set on the request
TC-SEC-010	Valid Token	Filter chain continues on a valid token
TC-SEC-011	Valid Token	Valid token does not trigger a 401 response
TC-SEC-012	Valid Token	JwtService.extractEmail() is called exactly once
TC-SEC-013	Invalid Token	Tampered token returns 401
TC-SEC-014	Invalid Token	Token for a deleted user returns 401
TC-SEC-015	Invalid Token	401 error body has application/json content type

---

Suite 12 — Spring Security Validation Tests (TS-SPRINGSEC-001)

File: `SpringSecurityValidationTest.java` | **Type:** @WebMvcTest + @WithMockUser | **Tests:** 10

TC ID	GROUP	WHAT IS VERIFIED
TC-SPRINGSEC-001	Headers	X-Content-Type-Options nosniff on all responses
TC-SPRINGSEC-002	Headers	X-Frame-Options DENY to prevent clickjacking
TC-SPRINGSEC-003	Headers	X-XSS-Protection header is present

TC ID	GROUP	WHAT IS VERIFIED
TC-SPRINGSEC-004	Headers	Security header confirmed via WebConfig interceptor
TC-SPRINGSEC-005	Headers	nosniff value confirmed on all responses
TC-SPRINGSEC-006	Mock User	USER role endpoint access is not 401
TC-SPRINGSEC-007	Mock User	USER role on goal engine is not 401
TC-SPRINGSEC-008	Mock User	GET requests do not return 415
TC-SPRINGSEC-009	Mock User	CSRF is disabled so POST requests are not blocked with 403
TC-SPRINGSEC-010	Mock User	ADMIN role in test config is not 403

Suite 13 — Testcontainers Integration Tests (TS-TC-001)

File: `WealthWiseIntegrationTest.java`
Type: @SpringBootTest with RANDOM_PORT + Testcontainers PostgreSQL 16
Tests: 12 — Requires Docker Desktop to be running

```
mvn test -Dtest=WealthWiseIntegrationTest --no-transfer-progress
```

TC ID	WHAT IS VERIFIED
TC-TC-001	Docker container starts and accepts JDBC connections
TC-TC-002	Full Spring application context loads against real PostgreSQL
TC-TC-003	GET /api/auth/health returns 200 (full stack verified)
TC-TC-004	POST /signup returns 200 and JWT with a real database write
TC-TC-005	Duplicate email returns 400 (UNIQUE constraint on real PostgreSQL)
TC-TC-006	POST /signin returns 200 and JWT with real bcrypt comparison
TC-TC-007	Wrong password returns 401 and not 500
TC-TC-008	Protected endpoint without JWT returns 401
TC-TC-009	Saved user has riskProfile MODERATE and non-null createdAt
TC-TC-010	findByEmail works correctly on real PostgreSQL dialect
TC-TC-011	User B sees zero of User A's transactions (data isolation)
TC-TC-012	Scheme search endpoint is publicly accessible without a token

---

Suite 14 — Frontend Component Tests (TS-FE-001)

File: `components.test.jsx` | **Type:** Vitest + React Testing Library + jsdom | **Tests:** 30

TC ID	COMPONENT	WHAT IS VERIFIED
TC-FE-001 to TC-FE-010	ErrorBoundary	Children render normally; crash triggers fallback UI; refresh button is present; brand name visible
TC-FE-011 to TC-FE-016	Stats	Section renders; exactly 4 metric cards; all label texts (Modules, Assets, XIRR Accuracy, SIP Plans)
TC-FE-017 to TC-FE-022	WarmupOverlay	Overlay shown when cold start; hidden when warmed; brand name and tip text present

TC ID	COMPONENT	WHAT IS VERIFIED
TC-FE-023 to TC-FE-030	Utility Functions	formatCurrency handles null and NaN; formatPercent converts decimals; formatCompactNumber formats Crores and Lakhs

Suite 15 — MSW API Integration Tests (TS-INT-FE-001)

File: `integration.test.js` | **Type:** Vitest + MSW (msw/node) | **Tests:** 15

MSW intercepts all fetch() calls and returns mock responses matching the real Spring Boot API contract. No backend required.

TC ID	GROUP	WHAT IS VERIFIED
TC-INT-FE-001	API Contract	Monte Carlo response has pessimistic, likely, optimistic, probability
TC-INT-FE-002	API Contract	Monte Carlo probability is in range [0, 1]
TC-INT-FE-003	API Contract	Deterministic response has boolean onTrack
TC-INT-FE-004	API Contract	Deterministic sensitivity array has exactly 3 items
TC-INT-FE-005	API Contract	requiredSip = currentSip + sipGap mathematical invariant
TC-INT-FE-006	API Contract	currentSipEnough is a boolean type
TC-INT-FE-007	API Contract	riskProfile is CONSERVATIVE, MODERATE, or AGGRESSIVE
TC-INT-FE-008	API Contract	Scheme search response content is an array
TC-INT-FE-009	API Contract	Wrong credentials return status 401
TC-INT-FE-010	API Contract	Valid signup returns a JWT token string
TC-INT-FE-011	Error Scenarios	Network error causes fetch to reject
TC-INT-FE-012	Error Scenarios	Server 500 returns a JSON error object
TC-INT-FE-013	Error Scenarios	Unknown endpoint returns 404
TC-INT-FE-014	Error Scenarios	100ms API delay is handled gracefully
TC-INT-FE-015	Error Scenarios	Missing password field returns 400

Visual Regression Testing — Storybook and Chromatic

Storybook isolates components and renders them in a browser sandbox. Chromatic takes snapshots and detects pixel-level visual changes automatically when code is updated.

FILE	COMPONENT	STORY ID	STATE TESTED
ErrorBoundary.stories.jsx	ErrorBoundary	TC-VR-001	Normal — children render correctly
		TC-VR-002	Crashed — fallback UI with refresh button shown
Stats.stories.jsx	Stats	TC-VR-003	Desktop — all 4 metric cards
		TC-VR-004	Mobile 375px — responsive stacked layout
		TC-VR-005	Tablet — 2-column responsive grid
WarmupOverlay.stories.jsx	WarmupOverlay	TC-VR-006	Cold start — overlay displayed
		TC-VR-007	Warmed up — overlay hidden, app visible
		TC-VR-008	Cold start detected but backend already warm

```
npm run storybook     # Dev server at http://localhost:6006
npx chromatic         # Publish to Chromatic for automated visual diff
```

Database Testing — PostgreSQL SQL Scripts

File: `backend/src/test/sql/schema_validation.sql`

```
psql -h <host> -U postgres -d <database> -f backend/src/test/sql/schema_validation.sql
```

TC ID	CATEGORY	TEST DESCRIPTION
TC-DB-001	Table Existence	users table exists in the public schema
TC-DB-002	Table Existence	transactions table exists
TC-DB-003	Table Existence	scheme_master table exists
TC-DB-004	Table Existence	nav_history table exists
TC-DB-005	Table Existence	investment_lots table exists
TC-DB-006	Column Existence	users.email column is present
TC-DB-007	Column Existence	users.risk_profile column is present
TC-DB-008	Column Existence	transactions.scheme_amfi_code column is present
TC-DB-009	Column Existence	nav_history.nav_date column is present
TC-DB-010	Column Existence	scheme_master.amfi_code column is present
TC-DB-011	Constraints	transactions has a UNIQUE constraint on transaction_ref
TC-DB-012	Constraints	users has a UNIQUE constraint on email
TC-DB-013	Constraints	nav_history has a UNIQUE constraint on amfi_code and nav_date
TC-DB-014	Defaults	users.risk_profile default value is MODERATE
TC-DB-015	Defaults	transactions.source default value is MANUAL
TC-DB-016	Data Isolation	PL/pgSQL script confirms User A transactions invisible to User B

End-to-End Testing — Playwright

File: `project/src/test/e2e/wealthwise.spec.js` | **Tests:** 15 cases

```
npx playwright test src/test/e2e/wealthwise.spec.js
npx playwright show-report
```

TC ID	GROUP	WHAT IS VERIFIED
TC-E2E-001 to TC-E2E-005	Landing Page	Title matches WealthWise; hero and stats sections visible; navbar present; no critical errors
TC-E2E-006 to TC-E2E-010	Auth Flow	Login modal opens; validation fires; wrong credentials show error; dashboard requires auth
TC-E2E-011 to TC-E2E-015	API Contract	Monte Carlo, Deterministic, Required SIP, Scheme Search, and Health endpoint all return correct shapes

Performance and Load Testing — k6

File: `backend/src/test/load/load-test.js`

```
k6 run --env BASE_URL=http://localhost:8080 backend/src/test/load/load-test.js
```

SCENARIO	VIRTUAL USERS	DURATION	PURPOSE
smoke_test	1 constant	30 seconds	Verify baseline at minimum load
load_test	0 to 10 to 20 ramp	2 minutes	Simulate realistic production traffic
stress_test	0 to 50 to 100 ramp	2 minutes	Find the application breaking point

GROUP ID	ENDPOINT	SLA TARGET
TC-PERF-001	GET /api/auth/health	Less than 200ms
TC-PERF-002	POST /api/learn/montecarlo	Less than 500ms
TC-PERF-003	POST /api/learn/deterministic	Less than 200ms
TC-PERF-004	GET /api/analytics/risk	Less than 1200ms
TC-PERF-005	GET /api/schemes/search	Less than 400ms

Security Scanning

OWASP Dependency-Check

```
mvn dependency-check:check --no-transfer-progress
# Report: target/dependency-check-report.html
# Build fails on any CVE with CVSS score 7 or above
```

SonarQube SAST

```
mvn sonar:sonar -Dsonar.projectKey=wealthwise-backend -Dsonar.host.url=https://sonarcloud.io
```

QUALITY GATE	REQUIRED THRESHOLD
Line Coverage (service classes)	70% or above
Blocker and Critical Issues	Zero
Technical Debt Ratio	Less than 5%
Duplicated Lines	Less than 3%

Known Defects and Observations

ID	SEVERITY	ISSUE	RECOMMENDED FIX
OBS-001	High	MethodArgumentNotValidException returns HTTP 500 instead of 400	Add @ExceptionHandler in GlobalExceptionHandler
OBS-002	Medium	Java 25 requires ByteBuddy experimental flag for Mockito	Upgrade to Spring Boot 3.3.x
OBS-003	Medium	No JaCoCo coverage gate enforced	Add jacoco-maven-plugin with 70% minimum

ID	SEVERITY	ISSUE	RECOMMENDED FIX
OBS-004	Medium	Testcontainers tests require Docker Desktop	Document as prerequisite in CI setup
OBS-005	Low	detectType with "Systematic Withdrawal Plan SWP" returns PURCHASE_SIP	Reorder keyword priority or use exclusive matching
OBS-006	Low	Spring Security sets X-XSS-Protection value to 0 in test context	Test for header existence not the exact value
OBS-007	Low	NavService final RestTemplate cannot be replaced by @InjectMocks	Test cache library directly; behaviours tested via Testcontainers

How to Run All Tests

```
# Backend — 235 tests (no Docker required)
mvn test -Dtest="!WealthWiseIntegrationTest" --no-transfer-progress

# Backend — Testcontainers only (Docker required)
mvn test -Dtest=WealthWiseIntegrationTest --no-transfer-progress

# Frontend — 45 tests
npm test

# Storybook — visual regression
npm run storybook

# Playwright — E2E
npx playwright test src/test/e2e/wealthwise.spec.js

# k6 — load tests
k6 run --env BASE_URL=http://localhost:8080 backend/src/test/load/load-test.js

# OWASP — dependency scan
mvn dependency-check:check --no-transfer-progress
```

Final Scorecard

METRIC	VALUE
Total Automated Tests	280
Backend Tests Passing	235
Frontend Tests Passing	45
Overall Pass Rate	100%
Backend Test Files	14 Java files across 6 packages
Testing Types Covered	14 distinct types
Named Test Suites	15
Storybook Story States	8 visual regression targets
Playwright E2E Cases	15
k6 Load Scenarios	3
SQL Schema Test Cases	16

WealthWise — Testing Documentation | Document 07 of 07
April 2026 | Bhuvan Nagesh | VTU Internship 2026 — Team 11
Backend: 235 of 235 | Frontend: 45 of 45 | Total: 280 of 280 | 100% Pass Rate

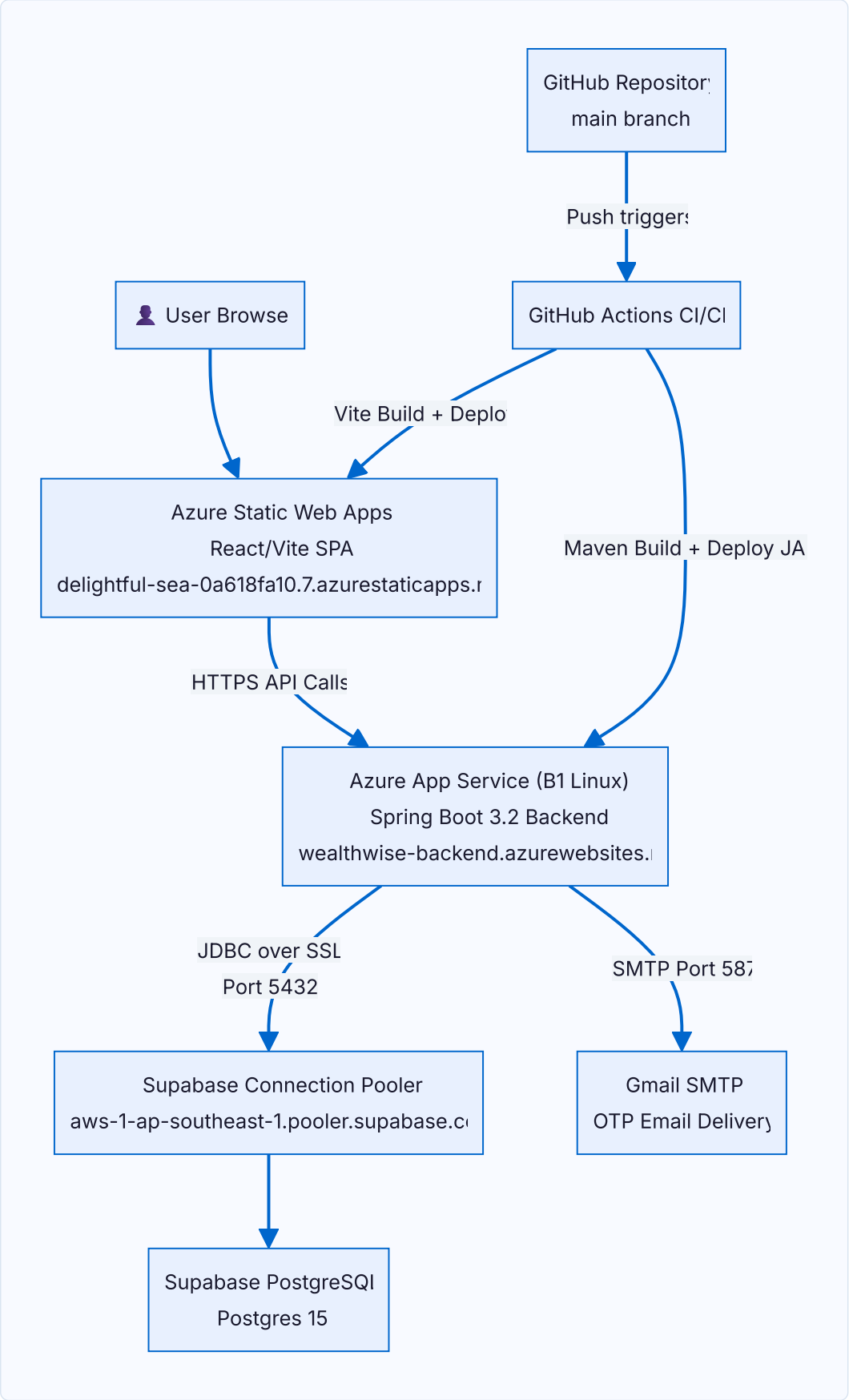
WealthWise — Azure Deployment Guide

1. Azure Architecture Overview

WealthWise is deployed on Microsoft Azure using a two-tier cloud architecture:

COMPONENT	AZURE SERVICE	TIER	URL
Backend API	Azure App Service (Linux)	B1 Basic	`https://wealthwise-backend.azurewebsites.net`
Frontend SPA	Azure Static Web Apps	Free	`https://delightful-sea-0a618fa10.7.azurestaticapps.net`
Database	Supabase PostgreSQL (via pooler)	Free	`aws-1-ap-southeast-1.pooler.supabase.com`
CI/CD	GitHub Actions	Free	Triggered on every push to `main`

Architecture Diagram



2. Azure Resources

2.1 Resource Group

PROPERTY	VALUE
Name	`wealthwise-rg`
Region	Central India
Subscription	Pay-As-You-Go

2.2 App Service Plan

PROPERTY	VALUE
Name	`wealthwise-plan`
OS	Linux
SKU	B1 Basic (1 vCPU, 1.75 GB RAM)
Region	Central India
Monthly Cost	~~₹1,100~~ ($13 USD)

2.3 Backend — Azure App Service

PROPERTY	VALUE
App Name	`wealthwise-backend`
Runtime	Java 17 (`JAVA|17-java17`)
OS	Linux
URL	`https://wealthwise-backend.azurewebsites.net`
Health Endpoint	`GET /api/auth/health` → `{"status":"UP"}`
Always On	Enabled

2.4 Frontend — Azure Static Web Apps

PROPERTY	VALUE
App Name	`wealthwise-frontend`
Tier	Free
URL	`https://delightful-sea-0a618fa10.7.azurestaticapps.net`
Build	Vite (`npm run build`) in `/project`
Output	`/project/dist`
Region	Central US (Global CDN)

---

3. CI/CD Pipeline — GitHub Actions

3.1 Backend Deployment Workflow

File: `.github/workflows/azure-backend.yml`

Trigger: Push to `main` branch with changes in `backend/**`

```yaml
name: Deploy Backend to Azure

on:
  push:
    branches: [main]
    paths:
      - 'backend/**'
      - '.github/workflows/azure-backend.yml'
  workflow_dispatch:

env:
  AZURE_WEBAPP_NAME: wealthwise-backend
  JAVA_VERSION: '17'

permissions:
  id-token: write
  contents: read

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: 'maven'

      - name: Build with Maven
        run: mvn clean package -DskipTests
        working-directory: ./backend

      - name: Login to Azure
        uses: azure/login@v2
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: Deploy to Azure Web App
        uses: azure/webapps-deploy@v3
        with:
          app-name: wealthwise-backend
          package: backend/target/wealthwise-backend-0.0.1-SNAPSHOT.jar
```

Pipeline Steps:

1. Checkout repository
2. Set up JDK 17 (Eclipse Temurin) with Maven cache
3. Build JAR with `mvn clean package -DskipTests`
4. Authenticate to Azure using Service Principal (`AZURE_CREDENTIALS`)
5. Deploy JAR to Azure App Service via Zip Deploy

3.2 Frontend Deployment Workflow

File: `.github/workflows/azure-static-web-apps-delightful-sea-0a618fa10.yml`

Trigger: Push to `main` branch (all files)

Auto-generated by Azure when the Static Web App was created. Uses the `azure/static-web-apps-deploy@v1` action:

```yaml
name: Azure Static Web Apps CI/CD

on:
  push:
    branches: [main]
  pull_request:
    types: [opened, synchronize, reopened, closed]
    branches: [main]

jobs:
  build_and_deploy_job:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build And Deploy
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${{ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN_DELIGHTFUL_SEA_0A618FA10
}}
          action: "upload"
          app_location: "/project"
          output_location: "dist"
```

Pipeline Steps:

1. Checkout repository
2. Install npm dependencies (`npm ci`)
3. Run Vite build (`npm run build`) in `/project`
4. Deploy `/project/dist` to Azure Static Web Apps CDN

---

4. GitHub Secrets Required

The following secrets must be configured in **GitHub → Settings → Secrets and variables → Actions**:

SECRET NAME	PURPOSE	HOW TO GENERATE
`AZURE_CREDENTIALS`	Service Principal JSON for Azure login	`az ad sp create-for-rbac --json-auth`
`AZURE_STATIC_WEB_APPS_API_TOKEN_DELIGHTFUL_SEA_0A618FA10`	Static Web App deployment token	Auto-generated by Azure when SWA is linked to GitHub

AZURE_CREDENTIALS Format

```json
{
  "clientId": "<service-principal-client-id>",
  "clientSecret": "<service-principal-secret>",
  "subscriptionId": "<azure-subscription-id>",
  "tenantId": "<azure-tenant-id>",
  "activeDirectoryEndpointUrl": "https://login.microsoftonline.com",
  "resourceManagerEndpointUrl": "https://management.azure.com/",
  "activeDirectoryGraphResourceId": "https://graph.windows.net/",
  "sqlManagementEndpointUrl": "https://management.core.windows.net:8443/",
  "galleryEndpointUrl": "https://gallery.azure.com/",
  "managementEndpointUrl": "https://management.core.windows.net/"
}
```

---

5. Azure App Service Configuration

5.1 Environment Variables (Application Settings)

All backend secrets are stored in **Azure Portal → App Service → Configuration → Application Settings**:

SETTING NAME	VALUE	DESCRIPTION
`SPRING_PROFILES_ACTIVE`	`azure`	Activates `application-azure.properties`
`WEBSITES_PORT`	`8080`	Port Azure App Service routes traffic to
`DB_URL`	`jdbc:postgresql://aws-1-ap-southeast-1.pooler.supabase.com:5432/postgres?sslmode=require`	Supabase connection pooler JDBC URL
`DB_USERNAME`	`postgres.pkktouewaqhvnwahoktn`	Supabase pooler username (project-scoped)
`DB_PASSWORD`	`<supabase-password>`	Supabase database password
`JWT_SECRET`	`<min-32-char-secret>`	JWT signing key
`MAIL_USERNAME`	`<gmail-address>`	Gmail for OTP delivery
`MAIL_PASSWORD`	`<gmail-app-password>`	Gmail App Password
`CORS_ALLOWED_ORIGINS`	`https://delightful-sea-0a618fa10.7.azurestaticapps.net,http://localhost:5173`	Allowed frontend origins

5.2 Azure Spring Profile (application-azure.properties)

File: `backend/src/main/resources/application-azure.properties`

Key differences from the production Render profile:

SETTING	AZURE VALUE	REASON
`spring.datasource.hikari.initialization-fail-timeout`	`-1`	Don't crash if DB connects slowly on cold start
`spring.jpa.defer-datasource-initialization`	`true`	Allow app to boot before DB validates
`spring.datasource.hikari.connection-timeout`	`60000`	60s timeout (Azure container has more time)
`spring.main.lazy-initialization`	`true`	Reduce startup time
`server.port`	`${WEBSITES_PORT:8080}`	Reads Azure's injected port

```
# Azure-specific Spring Boot profile
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.defer-datasource-initialization=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

app.jwt.secret=${JWT_SECRET}
server.port=${WEBSITES_PORT:8080}

# HikariCP — don't fail on slow cold start
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-timeout=60000
spring.datasource.hikari.initialization-fail-timeout=-1

spring.main.lazy-initialization=true
spring.main.banner-mode=off
```

---

6. Frontend Configuration

6.1 Static Web Apps Config

File: `staticwebapp.config.json` (project root)

```
{
  "routes": [
    {
      "route": "/api/*",
      "allowedRoles": ["anonymous"]
    },
    {
      "route": "/*",
      "serve": "/index.html",
      "statusCode": 200
    }
  ],
  "navigationFallback": {
    "rewrite": "/index.html",
    "exclude": ["/assets/*", "*.{css,js,png,jpg,svg,ico}"]
  },
  "globalHeaders": {
    "Cache-Control": "no-cache",
    "X-Content-Type-Options": "nosniff",
    "X-Frame-Options": "DENY",
    "X-XSS-Protection": "1; mode=block"
  },
  "mimeTypes": {
    ".json": "application/json"
  }
}
```

6.2 Frontend Environment Variables (.env.azure)

File: `project/.env.azure`

```
VITE_API_BASE_URL=https://wealthwise-backend.azurewebsites.net/api
VITE_APP_ENV=production
```

> ***Note:*** *Vite reads* `.env.production` *during build. Ensure* `VITE_API_BASE_URL` *is set in the Static Web App environment variables in Azure Portal for it to be embedded at build time.*

---

7. Database — Supabase Connection Pooler

Azure App Service requires the Supabase **Connection Pooler** host instead of the direct database host, because:

- The direct host (`db.<project>.supabase.co`) can fail DNS resolution in some Azure regions
- The connection pooler (`aws-1-ap-southeast-1.pooler.supabase.com`) uses PgBouncer for efficient connection management

PROPERTY	DIRECT CONNECTION	POOLER (USED FOR AZURE)
Host	`db.pkktouewaqhvnwahoktn.supabase.co`	`aws-1-ap-southeast-1.pooler.supabase.com`
Port	`5432`	`5432`
Username	`postgres`	`postgres.pkktouewaqhvnwahoktn`
SSL	Required	Required

8. Deployment Steps (First-Time Setup)

Step 1: Install Azure CLI

```
winget install Microsoft.AzureCLI
az login
az account set --subscription "<subscription-id>"
```

Step 2: Create Azure Resources

```
# Create resource group
az group create --name wealthwise-rg --location "Central India"

# Create App Service Plan (B1 Linux)
az appservice plan create --name wealthwise-plan --resource-group wealthwise-rg \
  --is-linux --sku B1

# Create Backend Web App (Java 17)
az webapp create --name wealthwise-backend --resource-group wealthwise-rg \
  --plan wealthwise-plan --runtime "JAVA:17-java17"

# Create Frontend Static Web App (links to GitHub)
az staticwebapp create --name wealthwise-frontend --resource-group wealthwise-rg \
  --source "https://github.com/BhuvanNagesh/VTU_INTERN_2026_TEAM11_JAVA" \
  --branch main --app-location "/project" --output-location "dist" \
  --login-with-github
```

Step 3: Configure App Settings

```
az webapp config appsettings set --name wealthwise-backend \
  --resource-group wealthwise-rg --settings \
  SPRING_PROFILES_ACTIVE="azure" \
  WEBSITES_PORT="8080" \
  DB_URL="jdbc:postgresql://aws-1-ap-southeast-1.pooler.supabase.com:5432/postgres?sslmode=require" \
  DB_USERNAME="postgres.pkktouewaqhvnwahoktn" \
  DB_PASSWORD="<your-password>" \
  JWT_SECRET="<your-32-char-secret>" \
  MAIL_USERNAME="<your-gmail>" \
  MAIL_PASSWORD="<your-app-password>" \
  CORS_ALLOWED_ORIGINS="https://delightful-sea-0a618fa10.7.azurestaticapps.net,http://localhost:5173"
```

Step 4: Create GitHub Service Principal

```
az ad sp create-for-rbac --name "wealthwise-github-deploy" \
  --role contributor \
  --scopes /subscriptions/<subscription-id>/resourceGroups/wealthwise-rg \
  --json-auth
```

Copy the JSON output and save it as the GitHub secret `AZURE_CREDENTIALS`.

Step 5: Add GitHub Secrets

Go to **GitHub → Repository → Settings → Secrets and variables → Actions → New repository secret**:

1. Name: `AZURE_CREDENTIALS` → Paste the service principal JSON
2. The Static Web App token (`AZURE_STATIC_WEB_APPS_API_TOKEN_*`) is auto-added by Azure when the SWA is linked

Step 6: Push to Main

```
git push origin main
```

GitHub Actions will:

- Build the Spring Boot JAR with Maven on ubuntu-latest
- Deploy the JAR to Azure App Service (Zip Deploy)
- Build the Vite SPA and deploy to Azure Static Web Apps CDN

Step 7: Verify Deployment

```
# Test backend health
curl https://wealthwise-backend.azurewebsites.net/api/auth/health
# Expected: {"status":"UP"}

# Test frontend
# Open in browser: https://delightful-sea-0a618fa10.7.azurestaticapps.net
```

---

9. Monitoring & Logs

9.1 View Application Logs

```
# Stream live logs
az webapp log tail --name wealthwise-backend --resource-group wealthwise-rg

# Download all logs
az webapp log download --name wealthwise-backend \
  --resource-group wealthwise-rg --log-file logs.zip
```

9.2 Stream Logs in Azure Portal

1. Go to **Azure Portal → App Service → wealthwise-backend**
2. Under **Monitoring → Log stream**
3. Logs stream in real-time including Spring Boot output

9.3 Application Insights (Optional)

For advanced monitoring, connect Application Insights:

```
az monitor app-insights component create --app wealthwise-insights \
  --location "Central India" --resource-group wealthwise-rg

az webapp config appsettings set --name wealthwise-backend \
  --resource-group wealthwise-rg --settings \
  APPLICATIONINSIGHTS_CONNECTION_STRING="<connection-string>"
```

---

10. Troubleshooting — Azure Specific

Issue 1: Container timeout on startup (503)

Symptom: `Container did not start within expected time limit of 230s`

Cause: App crashes during startup, usually due to DB connection failure.

Fix:

- Check logs: `az webapp log tail --name wealthwise-backend --resource-group wealthwise-rg`
- Verify all env vars are set: `az webapp config appsettings list --name wealthwise-backend --resource-group wealthwise-rg`
- Ensure `spring.datasource.hikari.initialization-fail-timeout=-1` is in `application-azure.properties`

Issue 2: UnknownHostException for Supabase host

Symptom: `java.net.UnknownHostException: db.pkktouewaqhvnwahoktn.supabase.co`

Cause: Azure's DNS cannot resolve the direct Supabase host in some regions.

Fix: Switch to the Supabase Connection Pooler URL:

```
DB_URL=jdbc:postgresql://aws-1-ap-southeast-1.pooler.supabase.com:5432/postgres?sslmode=require
DB_USERNAME=postgres.<project-ref>
```

Issue 3: GitHub Actions deploy fails — "Publish profile is invalid"

Symptom: `Error: Deployment Failed, Error: Publish profile is invalid`

Cause: Using publish profile method instead of service principal.

Fix: Use `azure/login@v2` with `AZURE_CREDENTIALS` service principal secret instead of `AZURE_WEBAPP_PUBLISH_PROFILE`.

Issue 4: Frontend shows blank page or 404 on refresh

Symptom: Navigating directly to `/dashboard` returns 404.

Cause: SPA routing — Azure Static Web Apps needs a fallback rule.

Fix: Ensure `staticwebapp.config.json` has the navigation fallback:

```
{
  "navigationFallback": {
    "rewrite": "/index.html"
  }
}
```

Issue 5: CORS error in browser console

Symptom: `Access to XMLHttpRequest at 'https://wealthwise-backend...' from origin 'https://delightful-sea...' has been blocked by CORS policy`

Fix: Update `CORS_ALLOWED_ORIGINS` in App Service settings to include the frontend URL:

```
az webapp config appsettings set --name wealthwise-backend \
  --resource-group wealthwise-rg --settings \
  CORS_ALLOWED_ORIGINS="https://delightful-sea-0a618fa10.7.azurestaticapps.net,http://localhost:5173"
az webapp restart --name wealthwise-backend --resource-group wealthwise-rg
```

Issue 6: Build fails — "Could not find artifact"

Symptom: GitHub Actions fails at the Maven build step.

Cause: `pom.xml` or dependency resolution issue.

Fix: Add `mvn dependency:resolve` step before `package`, or check `pom.xml` for invalid dependencies.

Issue 7: App restarts losing state

Expected behavior. Azure App Service B1 is stateless. JWT tokens remain valid across restarts as long as `JWT_SECRET` doesn't change. All data persists in Supabase.

---

11. Cost Summary

RESOURCE	TIER	MONTHLY COST (INR)	MONTHLY COST (USD)
App Service Plan B1	Basic	~₹1,100	~$13
Azure Static Web Apps	Free	₹0	$0
Supabase PostgreSQL	Free (500 MB)	₹0	$0
GitHub Actions	Free (2,000 min/month)	₹0	$0
Total		**~₹1,100**	**~$13**

> ***Cost optimisation tip:*** *To reduce cost to ₹0, switch to Azure Static Web Apps Managed Functions for the backend (requires refactoring to Azure Functions), or use Render's free tier for the backend.*

---

12. Comparison: Azure vs Render Deployment

ASPECT	AZURE (CURRENT)	RENDER (PREVIOUS)
Backend Runtime	Azure App Service B1 Linux	Docker container (free tier)
Build	GitHub Actions (Maven on ubuntu-latest)	Docker build from Dockerfile
Cold Start	~30–60s (Always On enabled)	~90s (free tier spins down)
Cost	~$13/month	$0 (free tier)
Always On	✅ Yes (B1 tier)	❌ No (spins down after 15 min)
CI/CD	GitHub Actions (automatic)	Render auto-deploy from GitHub
Database	Supabase (pooler)	Supabase (direct)
Frontend	Azure Static Web Apps (CDN)	Render Static Site
SSL	✅ Included	✅ Included
Custom Domain	✅ Supported	✅ Supported (free)